

COMPLETE DSA MATHEMATICS HANDBOOK

Mathematical formulas, invariants, and pattern-recognition triggers for coding interviews, LeetCode, Codeforces, FAANG-style interviews, and system-level coding assessments.

HOW TO USE THIS HANDBOOK

Every card is written in the same interview-first format:

1. FORMULA - the exact formula or invariant.
2. CONCEPT - what it means in simple language.
3. INTUITION - why it works.
4. DSA CONNECTION - which data structures and algorithms use it.
5. INTERVIEW PROBLEMS - famous problems where it appears.
6. PATTERN RECOGNITION - trigger words that should make the formula appear in your mind.
7. VISUALIZATION - a small diagram or concrete example.
8. COMMON MISTAKES - bugs and traps candidates hit.
9. MEMORY TRICK - a quick recall shortcut.
10. IMPLEMENTATION IMPACT - how it changes brute force into an optimal approach.

FAST PATTERN TRIGGER INDEX

| If you see... | Think... |

|---|---|

"range sum", "subarray sum", "sum between i and j"	Prefix sums
"range add", "many updates then queries"	Difference array
"divisible by k", "same remainder", "multiple of k"	Prefix modulo + frequency map
"gcd", "coprime", "linear combination"	Euclidean algorithm / Bezout
"mod inverse", "divide under modulo"	Extended Euclid or Fermat
"count ways", "choose", "arrange"	Combinatorics
"balanced parentheses", "unique BSTs", "non-crossing"	Catalan numbers
"appears twice except one"	XOR cancellation
"power of two"	$n > 0 \ \&\& \ (n \ \& \ (n - 1)) == 0$
"kth bit", "subset state", "visited set"	Bit masks
"orientation", "left turn", "convex hull"	Cross product
"tree with n nodes"	Edges = $n - 1$
"DAG dependency order"	Topological sort invariant
"minimum possible maximum", "capacity", "answer exists?"	Binary search on answer
"overlapping subproblems"	DP recurrence
"divide by half each step"	Logarithms
"recurrence $T(n) = aT(n/b) + f(n)$ "	Master theorem

PART 1: CORE ALGEBRA

1. ARITHMETIC PROGRESSION: NTH TERM

1. FORMULA: $a_n = a_1 + (n - 1)d$.
2. CONCEPT: The nth value in a sequence where every step adds the same difference d.
3. INTUITION: Starting at a_1 , reaching the nth item requires $n - 1$ jumps, and each jump changes the value by d.
4. DSA CONNECTION: Arrays, arithmetic sequences, missing numbers, binary search on positions, simulation compression.
5. INTERVIEW PROBLEMS: Missing Number (LC 268), Arithmetic Slices (LC 413), Can Make Arithmetic Progression (LC 1502).
6. PATTERN RECOGNITION: If values increase by a constant gap, or positions map linearly to values, use AP.
7. VISUALIZATION: 2, 5, 8, 11 has $a_1 = 2$, $d = 3$; item 4 is $2 + 3 \cdot 3 = 11$.
8. COMMON MISTAKES: Using nd instead of $(n - 1)d$; mixing 0-indexed and 1-indexed formulas.
9. MEMORY TRICK: "First plus jumps"; there are one fewer jumps than items.
10. IMPLEMENTATION IMPACT: Replaces iterative stepping over all positions with $O(1)$ direct computation.

2. ARITHMETIC PROGRESSION: SUM

1. FORMULA: $S_n = n(a_1 + a_n) / 2 = n(2a_1 + (n - 1)d) / 2$.
2. CONCEPT: The total of equally spaced numbers.
3. INTUITION: Pair the first and last terms: each pair sums to $a_1 + a_n$, and there are $n / 2$ pairs.

4. DSA CONNECTION: Prefix sum shortcuts, missing number, checksum validation, loop counting.
5. INTERVIEW PROBLEMS: Missing Number (LC 268), Arrange Coins (LC 441), Minimum Moves to Equal Array Elements (LC 453).
6. PATTERN RECOGNITION: If a brute force loop adds $1 + 2 + \dots + n$ or evenly spaced values, replace with AP sum.
7. VISUALIZATION: $1 + 2 + 3 + 4 = (1 + 4) + (2 + 3) = 5 + 5 = 10$.
8. COMMON MISTAKES: Integer overflow in $n * (n + 1)$; cast to long before multiplying.
9. MEMORY TRICK: "Average times count"; average is $(\text{first} + \text{last}) / 2$.
10. IMPLEMENTATION IMPACT: Turns $O(n)$ summation into $O(1)$, often preventing TLE for n up to 10^9 .

3. STANDARD SUMMATIONS

1. FORMULA: $\sum_{i=1..n} i = n(n + 1)/2$; $\sum i^2 = n(n + 1)(2n + 1)/6$; $\sum i^3 = [n(n + 1)/2]^2$.
2. CONCEPT: Closed forms for common loop totals.
3. INTUITION: Linear sums pair symmetrically; square and cube sums come from polynomial telescoping.
4. DSA CONNECTION: Complexity analysis, counting pairs/triples, DP precomputation, math puzzles.
5. INTERVIEW PROBLEMS: Sum of Square Numbers (LC 633), Count Good Triplets variants, nested-loop analysis.
6. PATTERN RECOGNITION: If loop work grows as i , i^2 , or cumulative nested loops, use these sums.
7. VISUALIZATION: Nested loop for i in $1..n$: for j in $1..i$ does $1 + 2 + \dots + n$ operations.
8. COMMON MISTAKES: Treating $\sum i^2$ as $O(n^2)$ instead of exact $n(n+1)(2n+1)/6$; overflow.
9. MEMORY TRICK: 1, 2, 3 powers have formulas with increasing polynomial degree: 2, 3, 4.
10. IMPLEMENTATION IMPACT: Gives exact counts for optimization and allows direct formulas instead of loops.

4. GEOMETRIC PROGRESSION

1. FORMULA: $a_n = a_1 r^{(n - 1)}$ and $S_n = a_1(r^n - 1)/(r - 1)$ for $r \neq 1$.
2. CONCEPT: A sequence where each term multiplies by a constant ratio.
3. INTUITION: Every step scales the previous value, so repeated multiplication becomes an exponent.
4. DSA CONNECTION: Binary search halving, exponential search, hashing powers, tree levels, amortized growth.
5. INTERVIEW PROBLEMS: Pow(x,n) (LC 50), Capacity/halving problems, Count Complete Tree Nodes (LC 222).
6. PATTERN RECOGNITION: If a value doubles, halves, or multiplies by a fixed ratio, think GP.
7. VISUALIZATION: 1, 2, 4, 8, 16; level h in a binary tree has up to 2^h nodes.
8. COMMON MISTAKES: Forgetting $r = 1$ special case; floating precision with non-integer ratios.
9. MEMORY TRICK: "Repeated multiply becomes power."
10. IMPLEMENTATION IMPACT: Converts repeated multiplication loops into fast exponentiation or logarithmic reasoning.

5. TELESCOPING DIFFERENCE FORMULA

1. FORMULA: $\sum_{i=l..r} (b_i - b_{i-1}) = b_r - b_{l-1}$.
2. CONCEPT: Consecutive middle terms cancel.
3. INTUITION: Expanding gives $(b_l - b_{l-1}) + (b_{l+1} - b_l) + \dots$; every internal $+b_i$ cancels a $-b_i$.
4. DSA CONNECTION: Difference arrays, prefix sums, amortized analysis, potential functions.
5. INTERVIEW PROBLEMS: Range Addition (LC 370), Corporate Flight Bookings (LC 1109), Car Pooling (LC 1094).
6. PATTERN RECOGNITION: If updates affect intervals but final values are needed, store boundary changes.
7. VISUALIZATION: Add +5 on $[2,4]$: diff has +5 at 2 and -5 at 5; prefix carries the value through the interval.
8. COMMON MISTAKES: Forgetting to subtract at $r + 1$; array bounds at the final index.
9. MEMORY TRICK: "Start the rain at l , stop it after r ."
10. IMPLEMENTATION IMPACT: Turns $O(qn)$ range updates into $O(q + n)$.

6. EXPONENT LAWS

1. FORMULA: $a^m a^n = a^{(m+n)}$, $a^m / a^n = a^{(m-n)}$, $(a^m)^n = a^{(mn)}$, $a^0 = 1$.
2. CONCEPT: Exponents count repeated multiplication.
3. INTUITION: Multiplying powers concatenates multiplication chains; powering a power repeats the chain.
4. DSA CONNECTION: Fast power, rolling hash powers, complexity growth, binary lifting.
5. INTERVIEW PROBLEMS: Pow(x,n) (LC 50), Super Pow (LC 372), string hashing tasks.
6. PATTERN RECOGNITION: If repeated multiplication appears, group powers by exponent rules.
7. VISUALIZATION: $2^3 * 2^2 = (2*2*2)*(2*2) = 2^5$.
8. COMMON MISTAKES: Applying exponent laws to addition, such as $(a + b)^2 = a^2 + b^2$ which is false.
9. MEMORY TRICK: Same base: multiply means add exponents.
10. IMPLEMENTATION IMPACT: Enables $O(\log n)$ exponentiation instead of $O(n)$.

7. LOGARITHM LAWS

1. FORMULA: $\log_b(xy) = \log_b x + \log_b y$; $\log_b(x^k) = k \log_b x$; $\log_b x = \log_c x / \log_c b$.
2. CONCEPT: Logs answer "how many times do I multiply or divide by the base?"
3. INTUITION: A logarithm is the inverse of exponentiation, so product and power rules reverse exponent laws.
4. DSA CONNECTION: Binary search, heap height, balanced trees, divide and conquer, complexity analysis.
5. INTERVIEW PROBLEMS: Binary Search (LC 704), Search Insert Position (LC 35), Kth Smallest in BST (LC 230).
6. PATTERN RECOGNITION: If a search space is repeatedly divided by a constant, complexity is logarithmic.
7. VISUALIZATION: $n = 1024$; halving reaches 1 in 10 steps because $2^{10} = 1024$.
8. COMMON MISTAKES: Ignoring base in implementation when exact counts matter; in Big O, constant bases collapse.
9. MEMORY TRICK: "Log counts cuts."
10. IMPLEMENTATION IMPACT: Justifies replacing linear scan with binary search or balanced-tree operations.

8. FLOOR AND CEILING DIVISION

1. FORMULA: For integers $a, b > 0$, $\text{floor}(a/b) = a / b$; $\text{ceil}(a/b) = (a + b - 1) / b$.
2. CONCEPT: Ceiling division counts how many full buckets of size b are needed to cover a items.
3. INTUITION: Adding $b - 1$ pushes any non-zero remainder into the next bucket.
4. DSA CONNECTION: Binary search on answer, pagination, batching, partitioning arrays, rate/capacity problems.
5. INTERVIEW PROBLEMS: Koko Eating Bananas (LC 875), Minimum Limit of Balls in a Bag (LC 1760), Capacity to Ship Packages (LC 1011).
6. PATTERN RECOGNITION: If the problem asks "how many groups/operations of size k ?", use ceiling division.
7. VISUALIZATION: 10 items in boxes of 3: $(10 + 2) / 3 = 4$ boxes.
8. COMMON MISTAKES: Using floating point `Math.ceil` and losing precision; negative numbers require language-aware handling.
9. MEMORY TRICK: "Add one less than the bucket."
10. IMPLEMENTATION IMPACT: Makes feasibility checks $O(n)$ and exact inside binary search.

9. INCLUSION-EXCLUSION PRINCIPLE

1. FORMULA: $|A \cup B| = |A| + |B| - |A \cap B|$; for three sets, add singles, subtract pairs, add triple intersection.
2. CONCEPT: Count everything, then remove double-counted overlap.
3. INTUITION: Elements in both sets are counted once from A and once from B , so subtract one copy.
4. DSA CONNECTION: Counting, bitmask enumeration, combinatorics, DP over sets, probability.
5. INTERVIEW PROBLEMS: Numbers With Repeated Digits (LC 1012), Vowel Strings counting variants, ugly number counting variants.
6. PATTERN RECOGNITION: If categories overlap and "or" appears, use inclusion-exclusion.
7. VISUALIZATION: Students who know Java or Python = Java + Python - both.
8. COMMON MISTAKES: Forgetting higher-order intersections; subtracting disjoint groups unnecessarily.
9. MEMORY TRICK: "Add odds, subtract evens" by intersection size.
10. IMPLEMENTATION IMPACT: Converts complex overcounted enumeration into direct counting or bitmask iteration.

10. MATHEMATICAL INDUCTION AND LOOP INVARIANTS

1. FORMULA: Prove $P(0)$ true, then prove $P(k) \rightarrow P(k+1)$; therefore $P(n)$ is true for all $n \geq 0$.
2. CONCEPT: A correctness proof for recursive algorithms and loops.
3. INTUITION: If the first domino falls and every domino knocks the next, all dominoes fall.
4. DSA CONNECTION: Recursion, DP, binary search invariants, greedy exchange arguments, tree algorithms.
5. INTERVIEW PROBLEMS: Merge Sort correctness, Binary Search correctness, Climbing Stairs (LC 70), House Robber (LC 198).
6. PATTERN RECOGNITION: If correctness depends on "after every iteration this remains true", state an invariant.
7. VISUALIZATION: Prefix loop invariant: after processing index i , the answer is correct for `arr[0..i]`.
8. COMMON MISTAKES: Proving only the sample case; not proving initialization, maintenance, and termination.
9. MEMORY TRICK: "Start, stay, stop": initialize, maintain, conclude.
10. IMPLEMENTATION IMPACT: Prevents off-by-one errors and makes binary search / DP transitions reliable.

11. ABSOLUTE DIFFERENCE AND TRIANGLE INEQUALITY

1. FORMULA: $|a - b| \geq 0$; $|a - c| \leq |a - b| + |b - c|$.

2. CONCEPT: Direct distance is never longer than going through an intermediate point.
3. INTUITION: Moving from a to c via b cannot beat the straight one-dimensional distance.
4. DSA CONNECTION: Greedy, two pointers, sorting by distance, median minimization, shortest paths.
5. INTERVIEW PROBLEMS: Minimum Moves to Equal Array Elements II (LC 462), 3Sum Closest (LC 16), Assign Cookies (LC 455).
6. PATTERN RECOGNITION: If the objective sums absolute distances, sort and look for medians or two pointers.
7. VISUALIZATION: On a line, 1 -> 7 has distance 6; going 1 -> 3 -> 7 gives $2 + 4 = 6$.
8. COMMON MISTAKES: Using mean instead of median for absolute-distance minimization.
9. MEMORY TRICK: "Absolute wants median; squares want mean."
10. IMPLEMENTATION IMPACT: Reduces search over all targets to a median or sorted greedy strategy.

12. MINIMUM AND MAXIMUM IDENTITIES

1. FORMULA: $\max(a,b) = (a + b + |a - b|)/2$; $\min(a,b) = (a + b - |a - b|)/2$.
2. CONCEPT: The absolute difference separates the larger and smaller values.
3. INTUITION: $a + b$ contains both numbers; adding the gap selects the larger twice, subtracting selects the smaller twice.
4. DSA CONNECTION: Math puzzles, branchless logic, DP transitions, interval merging.
5. INTERVIEW PROBLEMS: Best Time to Buy and Sell Stock (LC 121), max/min DP variants, Kadane variants.
6. PATTERN RECOGNITION: If a formula can be simplified by separating larger and smaller terms, use min/max identities.
7. VISUALIZATION: $a=8, b=3$: $(11 + 5)/2 = 8$, $(11 - 5)/2 = 3$.
8. COMMON MISTAKES: Overflow from $a + b$; branchless formulas are usually less safe than `Math.max`.
9. MEMORY TRICK: "Plus gap is max, minus gap is min."
10. IMPLEMENTATION IMPACT: Helps derive greedy formulas, though normal code should prefer clear min/max calls.

PART 2: PREFIX SUM MATHEMATICS

13. PREFIX SUM DEFINITION

1. FORMULA: $\text{pref}[0] = 0$; $\text{pref}[i + 1] = \text{pref}[i] + \text{arr}[i]$.
2. CONCEPT: $\text{pref}[i]$ stores the sum before index i .
3. INTUITION: Every prefix extends the previous prefix by one new element.
4. DSA CONNECTION: Arrays, range queries, sliding window, DP, hash maps.
5. INTERVIEW PROBLEMS: Range Sum Query Immutable (LC 303), Running Sum (LC 1480).
6. PATTERN RECOGNITION: If many queries ask for sums over intervals, precompute prefixes.
7. VISUALIZATION: $\text{arr}=[2,4,1]$; $\text{pref}=[0,2,6,7]$.
8. COMMON MISTAKES: Using $\text{pref}[i]$ as including $\text{arr}[i]$ in one place and excluding it in another.
9. MEMORY TRICK: pref length is $n + 1$; index means "items consumed."
10. IMPLEMENTATION IMPACT: Turns each range sum from $O(\text{length})$ into $O(1)$ after $O(n)$ preprocessing.

14. RANGE SUM FROM PREFIX

1. FORMULA: $\text{sum}(l,r) = \text{pref}[r + 1] - \text{pref}[l]$ for inclusive 0-indexed $[l,r]$.
2. CONCEPT: Remove the sum before l from the sum before $r + 1$.
3. INTUITION: The prefix to r contains unwanted left part plus target interval; subtraction isolates the interval.
4. DSA CONNECTION: Range queries, subarray optimization, immutable query structures.
5. INTERVIEW PROBLEMS: Range Sum Query (LC 303), Product of Array Except Self (LC 238) via prefix/suffix idea.
6. PATTERN RECOGNITION: If query asks "between l and r inclusive", use two prefix values.
7. VISUALIZATION: $\text{arr}=[3,1,4,2]$, $\text{pref}=[0,3,4,8,10]$; $\text{sum}[1,3] = 10 - 3 = 7$.
8. COMMON MISTAKES: Writing $\text{pref}[r] - \text{pref}[l]$ and dropping $\text{arr}[r]$.
9. MEMORY TRICK: "Right endpoint needs plus one."
10. IMPLEMENTATION IMPACT: Enables $O(n + q)$ instead of $O(nq)$.

15. SUBARRAY SUM EQUALS TARGET

1. FORMULA: $\text{sum}(j+1..i) = \text{pref}[i] - \text{pref}[j]$; need $\text{pref}[j] = \text{pref}[i] - \text{target}$.
2. CONCEPT: A previous prefix determines whether the current index closes a target-sum subarray.
3. INTUITION: If removing an earlier prefix leaves exactly target, the interval between them is valid.
4. DSA CONNECTION: HashMap, prefix sum, arrays with negative numbers.
5. INTERVIEW PROBLEMS: Subarray Sum Equals K (LC 560), Path Sum III (LC 437).
6. PATTERN RECOGNITION: If subarray sums can include negative values, sliding window may fail; use prefix + map.
7. VISUALIZATION: Current prefix 10, target 6; if old prefix 4 exists, middle sum is 6.
8. COMMON MISTAKES: Forgetting initial frequency $\text{freq}[0] = 1$; using set when counts are needed.
9. MEMORY TRICK: "Current minus target must have happened before."
10. IMPLEMENTATION IMPACT: Reduces $O(n^2)$ subarray enumeration to $O(n)$.

16. PREFIX MODULO REMAINDER

1. FORMULA: $(\text{pref}[i] - \text{pref}[j]) \% k = 0$ iff $\text{pref}[i] \% k = \text{pref}[j] \% k$.
2. CONCEPT: Two prefixes with the same remainder differ by a multiple of k .
3. INTUITION: Congruent numbers have a difference divisible by the modulus.
4. DSA CONNECTION: Prefix sums, HashMap frequencies, modular arithmetic.
5. INTERVIEW PROBLEMS: Continuous Subarray Sum (LC 523), Subarray Sums Divisible by K (LC 974).
6. PATTERN RECOGNITION: Words like divisible, remainder, multiple, modulo, or k with subarrays should trigger this.
7. VISUALIZATION: Prefix remainders: 1, 3, 1; the two 1s enclose a sum divisible by k .
8. COMMON MISTAKES: Not normalizing negative remainders with $(\text{rem} \% k + k) \% k$.
9. MEMORY TRICK: "Same remainder means divisible gap."
10. IMPLEMENTATION IMPACT: Converts $O(n^2)$ divisibility checks into $O(n)$ remainder counting.

17. COUNT SUBARRAYS DIVISIBLE BY K

1. FORMULA: If a remainder appears c times, it contributes $C(c,2) = c(c-1)/2$ subarrays.
2. CONCEPT: Any pair of equal prefix remainders forms a divisible subarray.
3. INTUITION: Choose two prefix positions with the same remainder; their difference is divisible by k .
4. DSA CONNECTION: HashMap, combinatorics, prefix modulo.
5. INTERVIEW PROBLEMS: Subarray Sums Divisible by K (LC 974).
6. PATTERN RECOGNITION: If asked for count, store frequencies rather than earliest index.
7. VISUALIZATION: Remainder 2 appears at prefix indices [0,3,6]; pairs are (0,3), (0,6), (3,6).
8. COMMON MISTAKES: Counting only existence; not adding current frequency before incrementing.
9. MEMORY TRICK: "Equal remainders pair up."
10. IMPLEMENTATION IMPACT: $O(n)$ counting instead of checking all $O(n^2)$ subarrays.

18. DIFFERENCE ARRAY FOR RANGE ADD

1. FORMULA: To add v on $[l,r]$: $\text{diff}[l] += v$; $\text{diff}[r+1] -= v$; final $\text{arr}[i] = \text{arr}[i-1] + \text{diff}[i]$.
2. CONCEPT: Mark where an interval effect starts and where it stops.
3. INTUITION: Prefixing the difference array carries active updates across their covered ranges.
4. DSA CONNECTION: Arrays, sweep line, range updates, offline processing.
5. INTERVIEW PROBLEMS: Range Addition (LC 370), Corporate Flight Bookings (LC 1109), Car Pooling (LC 1094).
6. PATTERN RECOGNITION: Many range updates with final array or point queries should trigger diff arrays.
7. VISUALIZATION: $+3$ on $[1,3]$: $\text{diff} [0,+3,0,0,-3]$; prefix gives $[0,3,3,3,0]$.
8. COMMON MISTAKES: Allocating only n slots when writing $r+1$; check bounds.
9. MEMORY TRICK: "Open bracket adds, closing bracket subtracts."
10. IMPLEMENTATION IMPACT: Turns $O(\text{updates} * \text{range length})$ into $O(\text{updates} + n)$.

19. SWEEP LINE EVENT DIFFERENCE

1. FORMULA: Active count at position x is $\text{sum events}[t]$ for all $t \leq x$.
2. CONCEPT: Intervals become start/end events.
3. INTUITION: When scanning left to right, starts increase the active set and ends decrease it.
4. DSA CONNECTION: Sorting, TreeMap, priority queue, interval problems.
5. INTERVIEW PROBLEMS: Meeting Rooms II (LC 253), My Calendar variants, Car Pooling (LC 1094).
6. PATTERN RECOGNITION: If intervals overlap and you need max simultaneous items, sweep events.
7. VISUALIZATION: Meeting $[10,20]$ adds $+1$ at 10, -1 at 20.
8. COMMON MISTAKES: Handling equal start/end times incorrectly; half-open intervals $[\text{start}, \text{end})$ avoid double counting.
9. MEMORY TRICK: "Timeline is a prefix sum over events."
10. IMPLEMENTATION IMPACT: Reduces pairwise interval overlap checks from $O(n^2)$ to $O(n \log n)$.

20. 2D PREFIX SUM

1. FORMULA: $\text{ps}[i+1][j+1] = \text{grid}[i][j] + \text{ps}[i][j+1] + \text{ps}[i+1][j] - \text{ps}[i][j]$.
2. CONCEPT: Store the sum of the rectangle from origin to each cell.
3. INTUITION: Add top and left rectangles, but their overlap was counted twice, so subtract it once.
4. DSA CONNECTION: Matrix range queries, image processing, DP grids.
5. INTERVIEW PROBLEMS: Range Sum Query 2D Immutable (LC 304), Max Sum of Rectangle No Larger Than K (LC 363).
6. PATTERN RECOGNITION: If many rectangle sum queries appear, build 2D prefix sums.
7. VISUALIZATION: Current rectangle = top block + left block + cell - top-left overlap.
8. COMMON MISTAKES: Forgetting the overlap subtraction; off-by-one when using $n+1$ by $m+1$ arrays.
9. MEMORY TRICK: "Add top and left, subtract the corner."
10. IMPLEMENTATION IMPACT: Makes rectangle sum $O(1)$ after $O(nm)$ preprocessing.

21. RECTANGLE SUM FORMULA

1. FORMULA: $\text{sum}(r1,c1,r2,c2) = \text{ps}[r2+1][c2+1] - \text{ps}[r1][c2+1] - \text{ps}[r2+1][c1] + \text{ps}[r1][c1]$.
2. CONCEPT: Use inclusion-exclusion to isolate a target rectangle.

3. INTUITION: Start with everything up to bottom-right, remove above and left, then add back the removed overlap.
4. DSA CONNECTION: 2D prefix, matrix DP, geometry-style grid counting.
5. INTERVIEW PROBLEMS: Range Sum Query 2D (LC 304), Count Square Submatrices variants.
6. PATTERN RECOGNITION: Query has two row bounds and two column bounds.
7. VISUALIZATION: Big rectangle - top strip - left strip + top-left corner.
8. COMMON MISTAKES: Reversing row/column; missing +1 on bottom/right because prefix is exclusive.
9. MEMORY TRICK: "Big minus two strips plus corner."
10. IMPLEMENTATION IMPACT: Avoids scanning $O(\text{area})$ cells for every query.

22. 2D DIFFERENCE ARRAY

1. FORMULA: Add v to rectangle $[r1..r2][c1..c2]$: $d[r1][c1] += v$, $d[r2+1][c1] -= v$, $d[r1][c2+1] -= v$, $d[r2+1][c2+1] += v$.
2. CONCEPT: Rectangle updates can be represented by four corners.
3. INTUITION: Prefixing rows and columns spreads the update across the rectangle; corner corrections stop the spread.
4. DSA CONNECTION: Grid updates, offline queries, image/matrix problems.
5. INTERVIEW PROBLEMS: Range Addition II variants, Stamp/painting grid problems.
6. PATTERN RECOGNITION: Many rectangle additions followed by final grid reconstruction.
7. VISUALIZATION: One corner starts the paint, two corners stop horizontal/vertical bleed, last corner restores the outside.
8. COMMON MISTAKES: Wrong signs at opposite corners; not allocating sentinel row/column.
9. MEMORY TRICK: "Plus, minus, minus, plus around the rectangle."
10. IMPLEMENTATION IMPACT: Converts $O(qnm)$ updates into $O(q + nm)$.

23. PREFIX XOR

1. FORMULA: $\text{xor}(l, r) = \text{px}[r + 1] \oplus \text{px}[l]$.
2. CONCEPT: XOR prefix works like sum prefix because equal values cancel.
3. INTUITION: $a \oplus a = 0$; everything before l appears twice and disappears.
4. DSA CONNECTION: Bit manipulation, arrays, tries, subarray XOR queries.
5. INTERVIEW PROBLEMS: XOR Queries of a Subarray (LC 1310), Count Triplets That Can Form Two Arrays of Equal XOR (LC 1442).
6. PATTERN RECOGNITION: If interval operation is XOR, use prefix XOR.
7. VISUALIZATION: $\text{px}[r+1] = \text{before} \oplus \text{target}$; XOR with before leaves target.
8. COMMON MISTAKES: Using addition-style subtraction; XOR inverse is XOR itself.
9. MEMORY TRICK: "XOR undoes itself."
10. IMPLEMENTATION IMPACT: Range XOR becomes $O(1)$ per query.

24. KADANE'S MAXIMUM SUBARRAY

1. FORMULA: $\text{bestEndingHere} = \max(x, \text{bestEndingHere} + x)$; $\text{answer} = \max(\text{answer}, \text{bestEndingHere})$.
2. CONCEPT: The best subarray ending at the current index either starts here or extends the previous best ending.
3. INTUITION: A negative previous contribution should be discarded because it only hurts future sums.
4. DSA CONNECTION: DP, arrays, greedy, prefix sums.
5. INTERVIEW PROBLEMS: Maximum Subarray (LC 53), Maximum Product Subarray (LC 152 variant).
6. PATTERN RECOGNITION: If asked for maximum contiguous sum, think Kadane.
7. VISUALIZATION: Running sum below zero is like carrying debt; restart after debt.
8. COMMON MISTAKES: Returning 0 for all-negative arrays when non-empty subarray is required.
9. MEMORY TRICK: "Extend if helpful, restart if harmful."
10. IMPLEMENTATION IMPACT: Replaces $O(n^2)$ all-subarray sums with $O(n)$.

PART 3: MODULAR ARITHMETIC

25. MODULAR NORMALIZATION

1. FORMULA: $\text{norm}(x, m) = ((x \% m) + m) \% m$.
2. CONCEPT: Convert any integer to a canonical remainder in $[0, m - 1]$.
3. INTUITION: Adding m does not change congruence, but fixes negative language remainders.
4. DSA CONNECTION: Prefix modulo, hash functions, circular arrays, number theory.
5. INTERVIEW PROBLEMS: Subarray Sums Divisible by K (LC 974), Robot bounded/circular movement variants.
6. PATTERN RECOGNITION: If values can be negative and $\%$ appears, normalize.
7. VISUALIZATION: $-1 \bmod 5$ should be 4; Java gives -1, so $(-1 + 5) \% 5 = 4$.
8. COMMON MISTAKES: Assuming $\%$ is mathematical modulo in every language.
9. MEMORY TRICK: "Mod, add mod, mod again."
10. IMPLEMENTATION IMPACT: Prevents missing HashMap buckets and wrong array indexes.

26. CONGRUENCE RELATION

1. FORMULA: $a \equiv b \pmod{m}$ iff m divides $a - b$.

2. CONCEPT: Two numbers have the same remainder under modulus m .
3. INTUITION: Numbers in the same remainder class differ by whole cycles of length m .
4. DSA CONNECTION: Prefix modulo, CRT, cycle detection, circular arrays.
5. INTERVIEW PROBLEMS: Continuous Subarray Sum (LC 523), Happy Number (LC 202) cycle reasoning.
6. PATTERN RECOGNITION: Same remainder, clock arithmetic, cyclic position, or divisibility by m .
7. VISUALIZATION: On a 12-hour clock, 15 and 3 land at the same position.
8. COMMON MISTAKES: Comparing raw values instead of normalized remainders.
9. MEMORY TRICK: "Same clock position."
10. IMPLEMENTATION IMPACT: Lets you group infinite integers into m finite buckets.

27. MODULAR ADDITION AND MULTIPLICATION

1. FORMULA: $(a + b) \% m = ((a \% m) + (b \% m)) \% m$; $(ab) \% m = ((a \% m)(b \% m)) \% m$.
2. CONCEPT: You can reduce operands before arithmetic.
3. INTUITION: Removing multiples of m from operands cannot affect the final remainder.
4. DSA CONNECTION: DP counts, combinatorics modulo $1e9+7$, rolling hash.
5. INTERVIEW PROBLEMS: Count Vowels Permutation (LC 1220), Number of Ways problems, Dice Roll DP.
6. PATTERN RECOGNITION: If answers are huge and problem says modulo, reduce at every step.
7. VISUALIZATION: Keep only the clock position after every move instead of tracking full laps.
8. COMMON MISTAKES: Applying modulo only at the end and overflowing long/int.
9. MEMORY TRICK: "Reduce early, reduce often."
10. IMPLEMENTATION IMPACT: Keeps DP/counting values bounded and safe.

28. FAST EXPONENTIATION

1. FORMULA: If n is even, $a^n = (a^{(n/2)})^2$; if odd, $a^n = a * a^{(n-1)}$.
2. CONCEPT: Exponentiation by squaring.
3. INTUITION: Binary representation of the exponent tells which squared powers are needed.
4. DSA CONNECTION: Number theory, modular inverse, matrix exponentiation, hashing.
5. INTERVIEW PROBLEMS: Pow(x,n) (LC 50), Super Pow (LC 372), Fibonacci via matrix exponentiation.
6. PATTERN RECOGNITION: Large exponent such as 10^9 or modulo power.
7. VISUALIZATION: $3^{13} = 3^{(8+4+1)} = 3^8 * 3^4 * 3$.
8. COMMON MISTAKES: Not handling negative exponents or Integer.MIN_VALUE overflow when negating.
9. MEMORY TRICK: "Square the base, halve the exponent."
10. IMPLEMENTATION IMPACT: Reduces $O(n)$ multiplication to $O(\log n)$.

29. PRIME NUMBER DEFINITION

1. FORMULA: p is prime iff its only positive divisors are 1 and p .
2. CONCEPT: A prime is an indivisible building block of integers greater than 1.
3. INTUITION: If $n = ab$, one factor is $\leq \sqrt{n}$; test divisors only up to $\sqrt{n}$.
4. DSA CONNECTION: Number theory, hashing, cryptography-style puzzles, sieve precomputation.
5. INTERVIEW PROBLEMS: Count Primes (LC 204), Ugly Number variants, Prime Arrangements (LC 1175).
6. PATTERN RECOGNITION: Divisors, factors, coprime, primality, or "smallest prime factor".
7. VISUALIZATION: For $36 = 4 * 9$, one factor is below $\sqrt{36}=6$.
8. COMMON MISTAKES: Treating 1 as prime; looping to $i*i \leq n$ with int overflow.
9. MEMORY TRICK: "Composite numbers show a small factor."
10. IMPLEMENTATION IMPACT: Trial division improves from $O(n)$ to $O(\sqrt{n})$.

30. SIEVE OF ERATOSTHENES

1. FORMULA: Mark multiples of each prime p starting at $p*p$.
2. CONCEPT: Precompute primality for all numbers up to n .
3. INTUITION: Smaller multiples of p were already marked by smaller factors.
4. DSA CONNECTION: Prime counting, factorization precompute, number theory DP.
5. INTERVIEW PROBLEMS: Count Primes (LC 204), Prime Subtraction Operation (LC 2601).
6. PATTERN RECOGNITION: Many prime queries up to a fixed limit.
7. VISUALIZATION: For $p=5$, start marking 25; 10,15,20 were marked by 2 or 3.
8. COMMON MISTAKES: Starting at $2p$ and doing extra work; wrong inclusive/exclusive bound for $< n$.
9. MEMORY TRICK: "Prime starts crossing at its square."
10. IMPLEMENTATION IMPACT: Builds primes up to n in $O(n \log \log n)$, far faster than repeated trial division.

31. GCD AND EUCLIDEAN ALGORITHM

1. FORMULA: $\gcd(a,b) = \gcd(b, a \% b)$ and $\gcd(a,0) = |a|$.
2. CONCEPT: The greatest common divisor is the largest number dividing both.
3. INTUITION: Any divisor of a and b also divides $a - qb$, so remainder preserves common divisors.
4. DSA CONNECTION: Number theory, fractions, graph/grid step normalization, array gcd.
5. INTERVIEW PROBLEMS: Greatest Common Divisor of Strings (LC 1071), Minimum Operations to Make Array GCD Equal to One variants.
6. PATTERN RECOGNITION: Common divisor, reduce fraction, repeated pattern length, coprime.
7. VISUALIZATION: $\gcd(48,18) \rightarrow \gcd(18,12) \rightarrow \gcd(12,6) \rightarrow 6$.

8. COMMON MISTAKES: Not using absolute values; recursion stack is usually fine but iterative is simple.

9. MEMORY TRICK: "Remainder keeps the gcd."

10. IMPLEMENTATION IMPACT: Computes gcd in $O(\log \min(a,b))$.

32. LCM

1. FORMULA: $\text{lcm}(a,b) = |a / \text{gcd}(a,b) * b|$.

2. CONCEPT: The smallest positive number divisible by both a and b.

3. INTUITION: $a*b$ contains both prime factorizations, but common factors are counted twice; divide by gcd once.

4. DSA CONNECTION: Periods, cycles, scheduling, CRT preconditions.

5. INTERVIEW PROBLEMS: Smallest Even Multiple (LC 2413), Replace Non-Coprime Numbers in Array (LC 2197).

6. PATTERN RECOGNITION: Synchronizing cycles or finding first common multiple.

7. VISUALIZATION: $\text{lcm}(12,18) = 12/6*18 = 36$.

8. COMMON MISTAKES: Multiplying before dividing and overflowing.

9. MEMORY TRICK: "LCM is product corrected by GCD."

10. IMPLEMENTATION IMPACT: Avoids scanning multiples one by one.

33. EXTENDED EUCLIDEAN ALGORITHM

1. FORMULA: Finds integers x,y such that $ax + by = \text{gcd}(a,b)$.

2. CONCEPT: The gcd can be expressed as a linear combination of the two numbers.

3. INTUITION: Back-substitute remainders from Euclid's algorithm until gcd is written using original a,b.

4. DSA CONNECTION: Modular inverse, Diophantine equations, CRT.

5. INTERVIEW PROBLEMS: Modular inverse in combinatorics, linear congruence competitive-programming tasks.

6. PATTERN RECOGNITION: "Find x such that $ax \equiv 1 \pmod m$ " or "integer solution to $ax + by = c$ ".

7. VISUALIZATION: $\text{gcd}(30,12)=6$; $6 = 30 - 2*12$.

8. COMMON MISTAKES: Sign errors during coefficient updates; forgetting inverse exists only when gcd is 1.

9. MEMORY TRICK: "Euclid gives gcd; extended Euclid gives the recipe."

10. IMPLEMENTATION IMPACT: Enables $O(\log m)$ modular division when inverse exists.

34. MODULAR INVERSE

1. FORMULA: $a^{-1} \pmod m$ is x such that $ax \equiv 1 \pmod m$; exists iff $\text{gcd}(a,m)=1$.

2. CONCEPT: Division under modulo is multiplication by an inverse.

3. INTUITION: From Bezout, if $ax + my = 1$, then $ax \equiv 1 \pmod m$.

4. DSA CONNECTION: Combinations modulo prime, probability modulo, CRT, hashing.

5. INTERVIEW PROBLEMS: Count Anagrams, nCr modulo prime, Codeforces combinatorics problems.

6. PATTERN RECOGNITION: If formula has division and final answer modulo, find modular inverse.

7. VISUALIZATION: Mod 7, inverse of 3 is 5 because $3*5 = 15 \equiv 1$.

8. COMMON MISTAKES: Using integer division under modulo; trying inverse when numbers are not coprime.

9. MEMORY TRICK: "Modulo division is multiply by undo."

10. IMPLEMENTATION IMPACT: Makes factorial-based combinations $O(1)$ after preprocessing.

35. FERMAT AND EULER THEOREMS

1. FORMULA: If p is prime and $a \% p \neq 0$, $a^{(p-1)} \equiv 1 \pmod p$; more generally $a^{\phi(m)} \equiv 1 \pmod m$ when $\text{gcd}(a,m)=1$.

2. CONCEPT: Powers cycle inside the multiplicative group modulo m.

3. INTUITION: Multiplying all non-zero residues by a coprime a permutes the same residue set.

4. DSA CONNECTION: Modular inverse, exponent reduction, number theory.

5. INTERVIEW PROBLEMS: Super Pow (LC 372), combinatorics modulo $1e9+7$.

6. PATTERN RECOGNITION: Prime modulus and inverse: use $a^{(p-2)} \pmod p$.

7. VISUALIZATION: Mod 5, powers of 2 cycle: 2,4,3,1.

8. COMMON MISTAKES: Applying Fermat when modulus is not prime or a divisible by p.

9. MEMORY TRICK: "Prime inverse exponent is p minus 2."

10. IMPLEMENTATION IMPACT: Computes inverse with fast power in $O(\log p)$.

36. EULER TOTIENT

1. FORMULA: If $n = \text{product } p_i^{e_i}$, then $\phi(n) = n * \text{product}(1 - 1/p_i)$.

2. CONCEPT: $\phi(n)$ counts integers in $[1,n]$ that are coprime to n.

3. INTUITION: For each prime factor p, remove multiples of p; inclusion-exclusion over prime factors gives the product.

4. DSA CONNECTION: Euler theorem, CRT, counting coprime pairs.

5. INTERVIEW PROBLEMS: Count coprime numbers, modular exponent reduction tasks.

6. PATTERN RECOGNITION: If a problem counts residues coprime to n, use phi.

7. VISUALIZATION: $\phi(12)=4$: 1,5,7,11.

8. COMMON MISTAKES: Forgetting distinct prime factors only in the product.
9. MEMORY TRICK: "Start with n, discount each prime factor."
10. IMPLEMENTATION IMPACT: Avoids checking gcd for every number when factorization is available.

37. CHINESE REMAINDER THEOREM

1. FORMULA: For pairwise coprime moduli m_i , system $x \equiv a_i \pmod{m_i}$ has a unique solution modulo $M = \text{product } m_i$.
2. CONCEPT: Independent remainder constraints combine into one modulo system.
3. INTUITION: Coprime cycles align exactly once over their product period.
4. DSA CONNECTION: Number theory, scheduling cycles, modular reconstruction.
5. INTERVIEW PROBLEMS: Bus schedule variants, Codeforces CRT tasks.
6. PATTERN RECOGNITION: Multiple congruences with different moduli.
7. VISUALIZATION: $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$; solution is $8 \pmod{15}$.
8. COMMON MISTAKES: Assuming CRT works unchanged for non-coprime moduli; must check compatibility.
9. MEMORY TRICK: "Coprime clocks meet once per product."
10. IMPLEMENTATION IMPACT: Replaces brute force search over huge ranges with modular construction.

38. LINEAR DIOPHANTINE EQUATION

1. FORMULA: $ax + by = c$ has integer solutions iff $\gcd(a,b)$ divides c .
2. CONCEPT: Only multiples of the gcd can be formed by integer combinations of a and b .
3. INTUITION: Every combination is divisible by gcd; Bezout proves every multiple of gcd is reachable.
4. DSA CONNECTION: Number theory, coin reachability, modular equations.
5. INTERVIEW PROBLEMS: Water and Jug Problem (LC 365), coin combination reachability variants.
6. PATTERN RECOGNITION: "Can measure exactly c using containers a and b " or $ax + by = \text{target}$.
7. VISUALIZATION: With jugs 3 and 5, gcd is 1, so any target up to capacity constraints may be reachable.
8. COMMON MISTAKES: Ignoring extra bounds like total capacity in jug problems.
9. MEMORY TRICK: "Reachable amounts are gcd multiples."
10. IMPLEMENTATION IMPACT: Turns state-space BFS into $O(\log n)$ math when only reachability is asked.

39. POLYNOMIAL ROLLING HASH

1. FORMULA: $H(s) = \sum s[i] \cdot \text{base}^i \pmod{\text{mod}}$; substring hash normalized by subtracting prefix and multiplying inverse power.
2. CONCEPT: Encode strings as modular polynomials.
3. INTUITION: Characters at different positions get different powers, so order affects the hash.
4. DSA CONNECTION: Strings, Rabin-Karp, duplicate substring, hash sets.
5. INTERVIEW PROBLEMS: Longest Duplicate Substring (LC 1044), Repeated DNA Sequences (LC 187).
6. PATTERN RECOGNITION: Need fast substring equality checks many times.
7. VISUALIZATION: "abc" becomes $a + b \cdot B + c \cdot B^2$.
8. COMMON MISTAKES: Hash collisions; use double hash or verify strings when required.
9. MEMORY TRICK: "String as a number in base B."
10. IMPLEMENTATION IMPACT: Substring comparisons go from $O(\text{length})$ to $O(1)$ average after preprocessing.

PART 4: COMBINATORICS

40. PRODUCT AND SUM RULES

1. FORMULA: Independent choices multiply: $\text{total} = a \cdot b$; disjoint alternatives add: $\text{total} = a + b$.
2. CONCEPT: Multiply stages, add alternatives.
3. INTUITION: For every first choice, there are all second choices; alternatives do not overlap.
4. DSA CONNECTION: Backtracking, counting DP, probability, combinatorics.
5. INTERVIEW PROBLEMS: Letter Combinations of a Phone Number (LC 17), Decode Ways (LC 91).
6. PATTERN RECOGNITION: "For each choice, choose another" means multiply; "either this or that" means add.
7. VISUALIZATION: 3 shirts and 2 pants make $3 \cdot 2 = 6$ outfits.
8. COMMON MISTAKES: Multiplying overlapping alternatives or adding independent stages.
9. MEMORY TRICK: "AND multiplies; OR adds" when cases are independent/disjoint.
10. IMPLEMENTATION IMPACT: Leads to recurrence counts without enumerating every object.

41. FACTORIAL

1. FORMULA: $n! = n \cdot (n-1) \cdot \dots \cdot 1$; $0! = 1$.
2. CONCEPT: Number of ways to order n distinct items.
3. INTUITION: First slot has n choices, next has $n-1$, and so on.
4. DSA CONNECTION: Permutations, backtracking, combinatorics modulo prime.
5. INTERVIEW PROBLEMS: Permutations (LC 46), Next Permutation (LC 31), Count Anagrams.
6. PATTERN RECOGNITION: If every object is used exactly once in an ordering, think factorial.
7. VISUALIZATION: ABC permutations: ABC, ACB, BAC, BCA, CAB, CBA = $3!$.
8. COMMON MISTAKES: Forgetting factorial growth explodes; $13!$ exceeds 32-bit int.

9. MEMORY TRICK: "Fill slots from left to right."

10. IMPLEMENTATION IMPACT: Helps identify when brute-force permutation generation is infeasible.

42. PERMUTATIONS

1. FORMULA: $P(n,k) = n!/(n-k)!$.

2. CONCEPT: Number of ordered selections of k items from n .

3. INTUITION: Fill k ordered slots: n choices, then $n-1$, down to $n-k+1$.

4. DSA CONNECTION: Backtracking, top- k arrangements, scheduling.

5. INTERVIEW PROBLEMS: Permutations II (LC 47), Beautiful Arrangement (LC 526).

6. PATTERN RECOGNITION: If order matters and not all items must be selected, use permutations.

7. VISUALIZATION: Choose 2 ordered letters from A,B,C: AB, AC, BA, BC, CA, CB.

8. COMMON MISTAKES: Using combinations when order matters.

9. MEMORY TRICK: "Permutation cares about position."

10. IMPLEMENTATION IMPACT: Bounds backtracking search and motivates pruning/memoization.

43. COMBINATIONS

1. FORMULA: $C(n,k) = n! / (k!(n-k)!)$.

2. CONCEPT: Number of unordered selections of k items from n .

3. INTUITION: Start with ordered selections, then divide by the $k!$ orders inside each chosen group.

4. DSA CONNECTION: Counting, binomial DP, subset generation, probability.

5. INTERVIEW PROBLEMS: Combinations (LC 77), Combination Sum variants, Pascal's Triangle (LC 118).

6. PATTERN RECOGNITION: If order does not matter and only membership matters, use combinations.

7. VISUALIZATION: From A,B,C, choosing 2 gives AB, AC, BC; AB and BA are the same choice.

8. COMMON MISTAKES: Not using symmetry $C(n,k)=C(n,n-k)$ to reduce computation.

9. MEMORY TRICK: "Choose ignores order."

10. IMPLEMENTATION IMPACT: Allows direct counts instead of enumerating all subsets.

44. BINOMIAL THEOREM

1. FORMULA: $(x + y)^n = \sum_{k=0..n} C(n,k)x^{n-k}y^k$.

2. CONCEPT: Expansion coefficients count how many ways each term appears.

3. INTUITION: In each of n factors choose either x or y ; choosing k y 's gives coefficient $C(n,k)$.

4. DSA CONNECTION: Combinatorics, probability, subset counts, polynomial DP.

5. INTERVIEW PROBLEMS: Unique Paths (LC 62), binomial probability/counting tasks.

6. PATTERN RECOGNITION: If each step has two types and order count matters only by number of one type, use binomial coefficients.

7. VISUALIZATION: $(x+y)^3 = xxx + xxy + xyx + yxx + xyy + yxy + yyx + yyy$.

8. COMMON MISTAKES: Treating coefficient as 1 for middle terms.

9. MEMORY TRICK: "Coefficient counts choices of y positions."

10. IMPLEMENTATION IMPACT: Turns path/count problems into $O(1)$ or $O(k)$ combinatorics after precompute.

45. PASCAL IDENTITY

1. FORMULA: $C(n,k) = C(n-1,k-1) + C(n-1,k)$.

2. CONCEPT: A combination either includes a chosen special item or it does not.

3. INTUITION: Split all k -subsets by whether they contain item n .

4. DSA CONNECTION: DP, Pascal triangle, binomial coefficients.

5. INTERVIEW PROBLEMS: Pascal's Triangle (LC 118), Unique Paths (LC 62).

6. PATTERN RECOGNITION: If count can split into "take" and "skip", use Pascal-style recurrence.

7. VISUALIZATION: Triangle row value is sum of two parents above it.

8. COMMON MISTAKES: Wrong base cases: $C(n,0)=C(n,n)=1$.

9. MEMORY TRICK: "Choose me or do not choose me."

10. IMPLEMENTATION IMPACT: Computes combinations with $O(nk)$ DP when factorial inverse is not available.

46. POWER SET COUNT

1. FORMULA: Number of subsets of n items is 2^n .

2. CONCEPT: Every item has two choices: included or excluded.

3. INTUITION: Product rule over n independent yes/no decisions.

4. DSA CONNECTION: Backtracking, bitmasking, subset DP.

5. INTERVIEW PROBLEMS: Subsets (LC 78), Partition Equal Subset Sum (LC 416), Maximum Product of Word Lengths (LC 318).

6. PATTERN RECOGNITION: If every element can be picked or skipped, expect 2^n .

7. VISUALIZATION: For $\{a,b\}$, subsets are $\{\}$, $\{a\}$, $\{b\}$, $\{a,b\}$.

8. COMMON MISTAKES: Trying subset enumeration for $n=40$ without meet-in-the-middle or DP.

9. MEMORY TRICK: "Each item is a bit."

10. IMPLEMENTATION IMPACT: Guides whether bitmask enumeration is feasible ($n \leq 20$ often).

47. CATALAN NUMBERS

1. FORMULA: $C_n = (1/(n+1)) * \text{binom}(2n,n)$ and $C_n = \sum_{i=0..n-1} C_i C_{n-1-i}$.
2. CONCEPT: Counts recursively nested non-crossing structures.
3. INTUITION: Pick a root/pair; left side has i elements and right side has $n-1-i$, independent Catalan subproblems.
4. DSA CONNECTION: DP, trees, parentheses, stack-valid sequences.
5. INTERVIEW PROBLEMS: Unique Binary Search Trees (LC 96), Generate Parentheses (LC 22).
6. PATTERN RECOGNITION: Balanced parentheses, unique BSTs, triangulations, non-crossing handshakes.
7. VISUALIZATION: For BST root r , smaller keys form left subtree and larger keys form right subtree.
8. COMMON MISTAKES: Confusing Catalan with 2^n ; Catalan is smaller but still grows fast.
9. MEMORY TRICK: "Catalan counts balanced/nested shapes."
10. IMPLEMENTATION IMPACT: Gives $O(n^2)$ DP for counts instead of enumerating all structures.

48. STARS AND BARS

1. FORMULA: Nonnegative solutions to $x_1 + \dots + x_k = n$ are $C(n+k-1, k-1)$.
2. CONCEPT: Distribute identical items into distinct boxes.
3. INTUITION: Arrange n stars and $k-1$ bars; bars split stars among boxes.
4. DSA CONNECTION: Counting DP, combinatorics, integer partitions with constraints.
5. INTERVIEW PROBLEMS: Coin distribution/counting variants, Codeforces distribution tasks.
6. PATTERN RECOGNITION: "Distribute n identical things among k groups" with nonnegative amounts.
7. VISUALIZATION: $*||$ means $(3,2,0)$ for 5 items into 3 boxes.
8. COMMON MISTAKES: Using this when items are distinct or boxes are identical; constraints need inclusion-exclusion/DP.
9. MEMORY TRICK: "Stars are items; bars are dividers."
10. IMPLEMENTATION IMPACT: Replaces recursive distribution enumeration with one combination.

49. MULTINOMIAL AND ANAGRAMS

1. FORMULA: Distinct permutations with counts $c_1..c_m$: $n! / (c_1! c_2! \dots c_m!)$.
2. CONCEPT: Repeated equal items make many permutations indistinguishable.
3. INTUITION: Start with $n!$ arrangements, then divide by internal swaps among identical copies.
4. DSA CONNECTION: String counting, HashMap frequencies, combinatorics modulo.
5. INTERVIEW PROBLEMS: Group Anagrams (LC 49 concept), Count Anagrams, permutation duplicate variants.
6. PATTERN RECOGNITION: If letters/items repeat and order matters, divide by duplicate factorials.
7. VISUALIZATION: "AAB" has $3!/2! = 3$: AAB, ABA, BAA.
8. COMMON MISTAKES: Forgetting modular inverse for division under modulo.
9. MEMORY TRICK: "Divide by swaps you cannot see."
10. IMPLEMENTATION IMPACT: Counts unique arrangements without generating duplicates.

50. DERANGEMENTS

1. FORMULA: $!n = (n-1)(!(n-1) + !(n-2))$; approximately $n!/e$.
2. CONCEPT: Number of permutations where no item stays in its original position.
3. INTUITION: Place item 1 into someone else's spot; that creates either a two-cycle or a longer chain.
4. DSA CONNECTION: DP, combinatorics, matching constraints.
5. INTERVIEW PROBLEMS: Find the Derangement of An Array (LC 634).
6. PATTERN RECOGNITION: "No fixed point", "no one gets own item".
7. VISUALIZATION: For 3 items, derangements are 231 and 312.
8. COMMON MISTAKES: Using $n! - n$ instead of inclusion-exclusion/DP.
9. MEMORY TRICK: "Nobody sits in their own seat."
10. IMPLEMENTATION IMPACT: Direct recurrence avoids enumerating all permutations.

51. PIGEONHOLE PRINCIPLE

1. FORMULA: Placing $n+1$ objects into n boxes forces some box to contain at least 2 objects.
2. CONCEPT: Too many objects for too few categories guarantees collision.
3. INTUITION: If every box had at most one object, total capacity would be only n .
4. DSA CONNECTION: Duplicate detection, cycle detection, hashing, proofs.
5. INTERVIEW PROBLEMS: Find the Duplicate Number (LC 287), Contains Duplicate variants.
6. PATTERN RECOGNITION: Values range has fewer possibilities than number of items.
7. VISUALIZATION: 367 people guarantee two share a birthday in a non-leap year.
8. COMMON MISTAKES: It proves existence, not location; still need an algorithm to find the duplicate.
9. MEMORY TRICK: "More pigeons than holes means sharing."
10. IMPLEMENTATION IMPACT: Justifies binary search on value or Floyd cycle for duplicate problems.

52. PROBABILITY BASICS

1. FORMULA: $P(A) = \text{favorable} / \text{total}$; $P(\text{not } A) = 1 - P(A)$; $P(A \cup B) = P(A) + P(B) - P(A \cap B)$.
2. CONCEPT: Probability is normalized counting.
3. INTUITION: Events partition the sample space; complements and inclusion-exclusion avoid overcounting.
4. DSA CONNECTION: Randomized algorithms, expected value, DP probability.

5. INTERVIEW PROBLEMS: Random Pick Index (LC 398), Random Pick with Weight (LC 528), New 21 Game (LC 837).
6. PATTERN RECOGNITION: Random choice, expected attempts, weighted sampling.
7. VISUALIZATION: Six-sided die: $P(\text{even}) = 3/6 = 1/2$.
8. COMMON MISTAKES: Assuming independence when events are dependent.
9. MEMORY TRICK: "Probability is count divided by universe."
10. IMPLEMENTATION IMPACT: Leads to prefix-weight sampling and probability DP instead of simulation.

53. EXPECTED VALUE LINEARITY

1. FORMULA: $E[X + Y] = E[X] + E[Y]$, even if X and Y are dependent.
2. CONCEPT: Average total equals sum of average contributions.
3. INTUITION: Summing outcomes and averaging can be swapped.
4. DSA CONNECTION: Randomized algorithms, hashing analysis, coupon/collision problems.
5. INTERVIEW PROBLEMS: RandomizedSet (LC 380) reasoning, reservoir sampling, expected comparisons in quicksort.
6. PATTERN RECOGNITION: Count total number of events by defining indicator variables.
7. VISUALIZATION: Expected number of heads in 10 coin flips is sum of ten $1/2$ indicators = 5.
8. COMMON MISTAKES: Thinking independence is required for linearity.
9. MEMORY TRICK: "Expectation adds even when variables do not behave."
10. IMPLEMENTATION IMPACT: Simplifies probabilistic analysis and avoids enumerating all outcomes.

PART 5: BIT MANIPULATION

54. XOR IDENTITIES

1. FORMULA: $x \oplus x = 0$; $x \oplus 0 = x$; XOR is associative and commutative.
2. CONCEPT: Equal values cancel under XOR.
3. INTUITION: Each bit toggled twice returns to its original value.
4. DSA CONNECTION: Bit manipulation, prefix XOR, missing/single number.
5. INTERVIEW PROBLEMS: Single Number (LC 136), Missing Number (LC 268), Single Number III (LC 260).
6. PATTERN RECOGNITION: If all elements appear twice except one or two, think XOR.
7. VISUALIZATION: $5 \oplus 7 \oplus 5 = 7$ because the two 5s cancel.
8. COMMON MISTAKES: XOR only cancels equal values an even number of times; triples need bit counting.
9. MEMORY TRICK: "XOR is toggle; toggle twice cancels."
10. IMPLEMENTATION IMPACT: Finds unique elements in $O(n)$ time and $O(1)$ space.

55. AND AND OR MASK MEANING

1. FORMULA: $x \& \text{mask}$ keeps selected bits; $x \mid \text{mask}$ forces selected bits to 1.
2. CONCEPT: AND filters bits; OR adds bits.
3. INTUITION: $1 \& \text{bit} = \text{bit}$, $0 \& \text{bit} = 0$; $1 \mid \text{bit} = 1$.
4. DSA CONNECTION: Bit masks, permissions, state compression, tries.
5. INTERVIEW PROBLEMS: Maximum Product of Word Lengths (LC 318), Bitwise ORs of Subarrays (LC 898).
6. PATTERN RECOGNITION: If a set has small universe size, represent membership with bits.
7. VISUALIZATION: Letters in word "abc" -> mask bits 000...0111.
8. COMMON MISTAKES: Using OR to test membership; use $(\text{mask} \& \text{bit}) \neq 0$.
9. MEMORY TRICK: "AND asks; OR adds."
10. IMPLEMENTATION IMPACT: Set operations become $O(1)$ word-level bit operations.

56. LEFT AND RIGHT SHIFT

1. FORMULA: $x \ll k = x * 2^k$; for nonnegative integers, $x \gg k = \text{floor}(x / 2^k)$.
2. CONCEPT: Shifts move bits by powers of two.
3. INTUITION: Binary place values are powers of two; moving left increases place value.
4. DSA CONNECTION: Bit masks, binary lifting, heap indexing, subset counts.
5. INTERVIEW PROBLEMS: Counting Bits (LC 338), Power of Two (LC 231), bitmask DP tasks.
6. PATTERN RECOGNITION: Need 2^k , kth bit, or divide/multiply by two.
7. VISUALIZATION: Binary 101 (5) shifted left one becomes 1010 (10).
8. COMMON MISTAKES: Signed right shift behavior for negative numbers; shift count overflow.
9. MEMORY TRICK: "Left grows, right shrinks."
10. IMPLEMENTATION IMPACT: Creates masks and state counts instantly.

57. POWER OF TWO TEST

1. FORMULA: $n > 0 \&\& (n \& (n - 1)) == 0$.
2. CONCEPT: Powers of two have exactly one set bit.
3. INTUITION: Subtracting 1 turns the single set bit into zeros above and ones below; AND clears everything.
4. DSA CONNECTION: Bit tricks, heap/capacity sizing, binary properties.
5. INTERVIEW PROBLEMS: Power of Two (LC 231), Power of Four (LC 342).
6. PATTERN RECOGNITION: If only one bit should be set, use $n \& (n-1)$.
7. VISUALIZATION: 8=1000, 7=0111; AND is 0000.

8. COMMON MISTAKES: Forgetting $n > 0$; zero also gives $(0 \& -1) == 0$.
9. MEMORY TRICK: "Power of two loses its only one."
10. IMPLEMENTATION IMPACT: $O(1)$ test instead of repeated division by 2.

58. LOWBIT

1. FORMULA: $\text{lowbit}(x) = x \& -x$.
2. CONCEPT: Extract the lowest set bit.
3. INTUITION: Two's complement $-x$ flips bits and adds 1, leaving only the rightmost 1 aligned.
4. DSA CONNECTION: Fenwick Tree, bit decomposition, subset iteration.
5. INTERVIEW PROBLEMS: Fenwick Tree range sum, Single Number III (LC 260).
6. PATTERN RECOGNITION: Need to isolate the first differing bit or Fenwick parent jump.
7. VISUALIZATION: $x=101100$; $x \& -x = 000100$.
8. COMMON MISTAKES: Confusing lowest set bit with index; use bit count or trailing zeros for index.
9. MEMORY TRICK: " x and negative x keeps the last 1."
10. IMPLEMENTATION IMPACT: Enables $O(\log n)$ Fenwick updates/queries.

59. CLEAR LOWEST SET BIT

1. FORMULA: $x = x \& (x - 1)$.
2. CONCEPT: Remove the lowest 1 bit from x .
3. INTUITION: $x - 1$ flips the lowest 1 to 0 and all lower zeros to 1; AND clears only that bit.
4. DSA CONNECTION: Popcount, subset loops, bit hacks.
5. INTERVIEW PROBLEMS: Number of 1 Bits (LC 191), Counting Bits (LC 338).
6. PATTERN RECOGNITION: Need to iterate set bits only.
7. VISUALIZATION: $110100 \rightarrow 110000 \rightarrow 100000 \rightarrow 000000$.
8. COMMON MISTAKES: Looping 32 times when number of set bits is much smaller.
9. MEMORY TRICK: " $n-1$ bites off the last 1."
10. IMPLEMENTATION IMPACT: Popcount runs $O(\text{number of set bits})$ instead of fixed bit width.

60. KTH BIT OPERATIONS

1. FORMULA: Set: $\text{mask} | (1 \ll k)$; clear: $\text{mask} \& \sim(1 \ll k)$; toggle: $\text{mask} \wedge (1 \ll k)$; test: $(\text{mask} \gg k) \& 1$.
2. CONCEPT: Use a one-bit mask to manipulate one position.
3. INTUITION: Bitwise operators affect only positions where the helper mask has 1.
4. DSA CONNECTION: Bitmask DP, visited sets, subset generation.
5. INTERVIEW PROBLEMS: Subsets (LC 78), Maximum Product of Word Lengths (LC 318), TSP-style DP.
6. PATTERN RECOGNITION: State contains yes/no flags for at most 20 to 30 items.
7. VISUALIZATION: To set bit 2: $0101 | 0100 = 0101$ if already set, else it becomes set.
8. COMMON MISTAKES: Missing parentheses: $1 \ll k + 1$ means $1 \ll (k+1)$ in many languages.
9. MEMORY TRICK: "Build a flashlight at bit k ."
10. IMPLEMENTATION IMPACT: Replaces boolean arrays/sets with compact integers and faster transitions.

61. SUBSET ENUMERATION

1. FORMULA: Iterate mask from 0 to $(1 \ll n) - 1$; bit i indicates whether item i is included.
2. CONCEPT: Every subset corresponds to one binary number.
3. INTUITION: n independent include/exclude choices form n bits, so there are 2^n masks.
4. DSA CONNECTION: Backtracking, meet-in-the-middle, bitmask DP.
5. INTERVIEW PROBLEMS: Subsets (LC 78), Partition to K Equal Sum Subsets (LC 698), Maximum Students Taking Exam (LC 1349).
6. PATTERN RECOGNITION: Small n and all subsets/states must be considered.
7. VISUALIZATION: For $n=3$, mask 101 means choose items 0 and 2.
8. COMMON MISTAKES: Overflow when $1 \ll n$ uses int and $n \geq 31$; use long if needed.
9. MEMORY TRICK: "A subset is a binary barcode."
10. IMPLEMENTATION IMPACT: Provides iterative enumeration and $O(1)$ membership checks.

62. SUBMASK ENUMERATION

1. FORMULA: for ($\text{sub} = \text{mask}$; $\text{sub} > 0$; $\text{sub} = (\text{sub} - 1) \& \text{mask}$).
2. CONCEPT: Iterate all subsets of a given mask.
3. INTUITION: Subtracting 1 moves to the next lower bit pattern; AND keeps only bits allowed by mask.
4. DSA CONNECTION: SOS DP, subset convolution, partition DP.
5. INTERVIEW PROBLEMS: Minimum Incompatibility (LC 1681), bitmask partitioning problems.
6. PATTERN RECOGNITION: Need to split a state mask into chosen subset and remainder.
7. VISUALIZATION: Mask 1011 generates 1011, 1010, 1001, 1000, 0011,
8. COMMON MISTAKES: Forgetting to process $\text{sub}=0$ separately when needed.
9. MEMORY TRICK: "Subtract then clamp to mask."
10. IMPLEMENTATION IMPACT: Enumerates valid submasks directly rather than scanning all 2^n masks each time.

63. BITMASK DP STATE COUNT

1. FORMULA: states = 2^n ; common transition cost $O(2^n * n)$ or $O(3^n)$ with submasks.
2. CONCEPT: Bitmask DP stores an answer for every subset.
3. INTUITION: Each item can be present or absent in a state.
4. DSA CONNECTION: DP, graph Hamiltonian paths, assignment, TSP.
5. INTERVIEW PROBLEMS: Shortest Path Visiting All Nodes (LC 847), Can I Win (LC 464), TSP variants.
6. PATTERN RECOGNITION: $n \leq 20$, visited-set state, all subsets.
7. VISUALIZATION: $dp[mask][last]$ means "best result after visiting exactly mask and ending at last."
8. COMMON MISTAKES: Trying bitmask DP with $n=30+$; memory explodes.
9. MEMORY TRICK: "Small n can become state bits."
10. IMPLEMENTATION IMPACT: Converts exponential recursion with repeated states into memoized 2^n states.

PART 6: GEOMETRY

64. EUCLIDEAN DISTANCE

1. FORMULA: $\text{dist}((x1,y1),(x2,y2)) = \sqrt{(x2-x1)^2 + (y2-y1)^2}$.
2. CONCEPT: Straight-line distance between two points.
3. INTUITION: Horizontal and vertical differences form a right triangle.
4. DSA CONNECTION: Geometry, nearest points, graph edge weights.
5. INTERVIEW PROBLEMS: K Closest Points to Origin (LC 973), Min Cost to Connect Points (LC 1584 variant with Manhattan).
6. PATTERN RECOGNITION: Coordinate points with "closest", "distance", "radius".
7. VISUALIZATION: From (0,0) to (3,4), distance is 5.
8. COMMON MISTAKES: Computing sqrt unnecessarily; compare squared distances to avoid floating error.
9. MEMORY TRICK: "Distance is Pythagoras on deltas."
10. IMPLEMENTATION IMPACT: Squared distance keeps comparisons $O(1)$, integer, and precise.

65. MIDPOINT

1. FORMULA: $\text{mid} = ((x1+x2)/2, (y1+y2)/2)$.
2. CONCEPT: Point halfway between two points.
3. INTUITION: Average each coordinate independently.
4. DSA CONNECTION: Geometry, binary search on coordinates, divide and conquer.
5. INTERVIEW PROBLEMS: Symmetry/reflection point problems, rectangle center checks.
6. PATTERN RECOGNITION: Need center, reflection, or pair symmetry.
7. VISUALIZATION: Midpoint of (2,4) and (8,10) is (5,7).
8. COMMON MISTAKES: Integer division losing .5; use doubled coordinates for exact hash keys.
9. MEMORY TRICK: "Midpoint is coordinate average."
10. IMPLEMENTATION IMPACT: Enables $O(n)$ symmetry checks by comparing paired sums instead of floats.

66. PYTHAGOREAN THEOREM

1. FORMULA: For a right triangle, $a^2 + b^2 = c^2$.
2. CONCEPT: Relates legs and hypotenuse.
3. INTUITION: The square on the hypotenuse has area equal to the sum of squares on the legs.
4. DSA CONNECTION: Distance, grid geometry, right-triangle counting.
5. INTERVIEW PROBLEMS: Valid Square (LC 593), Detect Squares (LC 213).
6. PATTERN RECOGNITION: Right angle, square validation, perpendicular distances.
7. VISUALIZATION: 3-4-5 triangle: $9 + 16 = 25$.
8. COMMON MISTAKES: Floating comparison after square root; compare squared lengths.
9. MEMORY TRICK: "Leg squares add to hypotenuse square."
10. IMPLEMENTATION IMPACT: Avoids expensive and imprecise square roots in geometry checks.

67. LINE SLOPE AND EQUATION

1. FORMULA: $\text{slope} = (y2-y1)/(x2-x1)$; line: $Ax + By + C = 0$.
2. CONCEPT: Slope describes steepness; standard form handles vertical lines.
3. INTUITION: Slope is rise over run; all points on a line satisfy one linear equation.
4. DSA CONNECTION: Geometry hashing, collinearity, line sweep.
5. INTERVIEW PROBLEMS: Max Points on a Line (LC 149).
6. PATTERN RECOGNITION: Collinear points or same line grouping.
7. VISUALIZATION: Points (0,0),(2,2),(3,3) all have slope 1.
8. COMMON MISTAKES: Using floating slopes as map keys; normalize $(dy/g, dx/g)$ with gcd.
9. MEMORY TRICK: "Slope is rise/run; hash as reduced pair."
10. IMPLEMENTATION IMPACT: Turns $O(n^3)$ collinearity into $O(n^2)$ grouping.

68. DOT PRODUCT

1. FORMULA: $u \cdot v = u_x v_x + u_y v_y = |u||v|\cos(\theta)$.
2. CONCEPT: Measures alignment between vectors.
3. INTUITION: Projection of one vector onto another scales by cosine of the angle.
4. DSA CONNECTION: Angles, projections, perpendicular checks, geometry optimization.

5. INTERVIEW PROBLEMS: Valid Square (LC 593), angle sorting geometry tasks.
6. PATTERN RECOGNITION: Need angle type: acute, right, obtuse, or projection.
7. VISUALIZATION: If $u \cdot v = 0$, vectors are perpendicular.
8. COMMON MISTAKES: Mistaking dot product for cross product orientation.
9. MEMORY TRICK: "Dot tells how much same direction."
10. IMPLEMENTATION IMPACT: Tests right angles in $O(1)$ without trigonometry.

69. CROSS PRODUCT AND ORIENTATION

1. FORMULA: $\text{cross}(b-a, c-a) = (b_x-a_x)(c_y-a_y) - (b_y-a_y)(c_x-a_x)$.
2. CONCEPT: Sign tells whether turn $a \rightarrow b \rightarrow c$ is left, right, or collinear.
3. INTUITION: Cross product is signed parallelogram area.
4. DSA CONNECTION: Convex hull, segment intersection, polygon area.
5. INTERVIEW PROBLEMS: Erect the Fence (LC 587), Line Reflection (LC 356).
6. PATTERN RECOGNITION: "clockwise", "counterclockwise", "left turn", "convex".
7. VISUALIZATION: Positive cross means c lies to the left of directed edge $a \rightarrow b$.
8. COMMON MISTAKES: Reversing argument order and flipping the sign.
9. MEMORY TRICK: "Cross sign is turn sign."
10. IMPLEMENTATION IMPACT: Enables robust integer geometry without computing angles.

70. TRIANGLE AREA

1. FORMULA: $\text{area} = |\text{cross}(b-a, c-a)| / 2$.
2. CONCEPT: Triangle area is half the parallelogram area.
3. INTUITION: Two copies of the triangle form a parallelogram spanned by the vectors.
4. DSA CONNECTION: Geometry, collinearity, polygon algorithms.
5. INTERVIEW PROBLEMS: Largest Triangle Area (LC 812), convex hull tasks.
6. PATTERN RECOGNITION: Three points and area/collinearity.
7. VISUALIZATION: If cross is 0, area is 0, so points are collinear.
8. COMMON MISTAKES: Losing .5 with integer division; store doubled area when possible.
9. MEMORY TRICK: "Triangle is half a cross."
10. IMPLEMENTATION IMPACT: $O(1)$ exact area comparisons using doubled area.

71. SHOELACE POLYGON AREA

1. FORMULA: $\text{area} = |\sum (x_i y_{i+1} - y_i x_{i+1})| / 2$.
2. CONCEPT: Computes area of a polygon from ordered vertices.
3. INTUITION: Decompose polygon into signed triangles from the origin; internal parts cancel.
4. DSA CONNECTION: Computational geometry, convex hull, lattice polygons.
5. INTERVIEW PROBLEMS: Polygon area tasks, Erect the Fence follow-ups.
6. PATTERN RECOGNITION: Ordered polygon vertices and area.
7. VISUALIZATION: Multiply diagonally down and subtract diagonally up, like tying shoelaces.
8. COMMON MISTAKES: Vertices must be in boundary order; unordered points do not work.
9. MEMORY TRICK: "Down products minus up products."
10. IMPLEMENTATION IMPACT: Computes polygon area in $O(n)$.

72. RECTANGLE AND CIRCLE AREA

1. FORMULA: Rectangle area $w \cdot h$; circle area πr^2 ; circumference $2\pi r$.
2. CONCEPT: Basic area formulas for geometric bounds.
3. INTUITION: Rectangle tiles by unit squares; circle area grows with square of radius.
4. DSA CONNECTION: Geometry, bounding boxes, spatial indexing.
5. INTERVIEW PROBLEMS: Rectangle Overlap (LC 836), Rectangle Area (LC 223), Random Point in Non-overlapping Rectangles (LC 497).
6. PATTERN RECOGNITION: Axis-aligned rectangles, radius constraints, overlap area.
7. VISUALIZATION: Overlap width is $\max(0, \min(r1, r2) - \max(l1, l2))$.
8. COMMON MISTAKES: Negative overlap dimensions; always clamp to zero.
9. MEMORY TRICK: "Overlap is intersection of x-intervals times y-intervals."
10. IMPLEMENTATION IMPACT: $O(1)$ rectangle overlap/area checks.

73. MANHATTAN AND CHEBYSHEV DISTANCE

1. FORMULA: Manhattan $|x1-x2| + |y1-y2|$; Chebyshev $\max(|dx|, |dy|)$.
2. CONCEPT: Manhattan counts grid moves in 4 directions; Chebyshev counts king moves in 8 directions.
3. INTUITION: Without diagonals, horizontal and vertical costs add; with diagonals, the larger delta dominates.
4. DSA CONNECTION: Grids, BFS heuristics, nearest point, transforms.
5. INTERVIEW PROBLEMS: Min Cost to Connect Points (LC 1584), Escape the Ghosts (LC 789).
6. PATTERN RECOGNITION: Grid movement with/without diagonals.
7. VISUALIZATION: From (0,0) to (3,4): Manhattan 7, Chebyshev 4.
8. COMMON MISTAKES: Using Euclidean distance for grid step counts.
9. MEMORY TRICK: "City blocks add; king moves max."
10. IMPLEMENTATION IMPACT: Provides exact $O(1)$ movement cost without BFS when no obstacles exist.

74. SEGMENT INTERSECTION

1. FORMULA: Segments intersect if orientations straddle: $\text{orient}(a,b,c) \cdot \text{orient}(a,b,d) \leq 0$ and $\text{orient}(c,d,a) \cdot \text{orient}(c,d,b) \leq 0$, plus bounding-box checks for collinear cases.
2. CONCEPT: Each segment's endpoints must lie on opposite sides of the other segment, or touch.
3. INTUITION: A crossing means the line through one segment separates the endpoints of the other.
4. DSA CONNECTION: Geometry, sweep line, collision detection.
5. INTERVIEW PROBLEMS: Rectangle/line intersection variants, computational geometry interviews.
6. PATTERN RECOGNITION: Need to know whether two line segments touch/cross.
7. VISUALIZATION: An X shape has endpoints alternating sides.
8. COMMON MISTAKES: Ignoring collinear overlap and endpoint touching.
9. MEMORY TRICK: "Both pairs must straddle."
10. IMPLEMENTATION IMPACT: $O(1)$ segment test used inside $O(n \log n)$ sweep algorithms.

75. CONVEX HULL TURN INVARIANT

1. FORMULA: In monotonic chain, while last three points make a non-left turn, pop the middle point.
2. CONCEPT: Convex hull boundary must keep consistent turns.
3. INTUITION: A point that creates an inward/right turn cannot be on the outer convex boundary.
4. DSA CONNECTION: Geometry, stack, sorting.
5. INTERVIEW PROBLEMS: Erect the Fence (LC 587).
6. PATTERN RECOGNITION: Need outer boundary enclosing all points.
7. VISUALIZATION: Rubber band around nails touches only convex outer nails.
8. COMMON MISTAKES: Handling collinear boundary points incorrectly depending on whether all boundary points are required.
9. MEMORY TRICK: "Hull stack only keeps outward turns."
10. IMPLEMENTATION IMPACT: Computes hull in $O(n \log n)$ due to sorting, $O(n)$ scan after sort.

PART 7: TREES AND GRAPHS MATHEMATICS

76. TREE EDGE COUNT

1. FORMULA: A tree with n nodes has $n - 1$ edges.
2. CONCEPT: A connected acyclic graph has exactly one fewer edge than nodes.
3. INTUITION: Start with one node and add each new node with exactly one connecting edge.
4. DSA CONNECTION: Trees, DFS, BFS, Union Find, graph validation.
5. INTERVIEW PROBLEMS: Graph Valid Tree (LC 261), Redundant Connection (LC 684).
6. PATTERN RECOGNITION: If graph should be a tree, check connected and edges $n-1$.
7. VISUALIZATION: 5 nodes in a chain need 4 links.
8. COMMON MISTAKES: Edge count alone is not sufficient; graph also must be connected.
9. MEMORY TRICK: "Tree edges trail nodes by one."
10. IMPLEMENTATION IMPACT: Quickly rejects invalid graph before DFS/DSU.

77. BINARY TREE LEVEL COUNTS

1. FORMULA: Maximum nodes at level h is 2^h if root level is 0.
2. CONCEPT: Each level can double the previous level.
3. INTUITION: Every node has at most two children.
4. DSA CONNECTION: Binary trees, heaps, BFS level order.
5. INTERVIEW PROBLEMS: Binary Tree Level Order Traversal (LC 102), Maximum Depth (LC 104).
6. PATTERN RECOGNITION: Complete/perfect binary tree or level-size reasoning.
7. VISUALIZATION: Levels: 1, 2, 4, 8,
8. COMMON MISTAKES: Mixing height as nodes count vs edge count.
9. MEMORY TRICK: "Binary doubles by level."
10. IMPLEMENTATION IMPACT: Bounds memory for BFS queues and heap sizes.

78. PERFECT BINARY TREE NODE COUNT

1. FORMULA: Height h by edges has nodes $2^{(h+1)} - 1$; leaves 2^h .
2. CONCEPT: A perfect tree has every level completely full.
3. INTUITION: Sum levels $1 + 2 + \dots + 2^h$, a geometric series.
4. DSA CONNECTION: Trees, heaps, segment trees.
5. INTERVIEW PROBLEMS: Count Complete Tree Nodes (LC 222), Populating Next Right Pointers (LC 116).
6. PATTERN RECOGNITION: Full levels or complete/perfect tree constraints.
7. VISUALIZATION: Height 2: $1 + 2 + 4 = 7$.
8. COMMON MISTAKES: Using $2^h - 1$ with inconsistent height definition.
9. MEMORY TRICK: "Full binary tree nodes are next power of two minus one."
10. IMPLEMENTATION IMPACT: Allows subtree counts in $O(1)$ once height equality proves perfection.

79. HEAP ARRAY INDEXING

1. FORMULA: 0-indexed: parent $(i - 1)/2$, left $2i + 1$, right $2i + 2$.

2. CONCEPT: Complete binary tree stored level-order in an array.
3. INTUITION: Each level fills left to right, so children positions follow arithmetic offsets.
4. DSA CONNECTION: Heap, priority queue, heap sort.
5. INTERVIEW PROBLEMS: Kth Largest Element (LC 215), Merge k Sorted Lists (LC 23), Top K Frequent Elements (LC 347).
6. PATTERN RECOGNITION: Complete tree with priority ordering.
7. VISUALIZATION: Index 0 has children 1,2; index 1 has 3,4.
8. COMMON MISTAKES: Mixing 1-indexed formulas ($\text{parent}=i/2$, children $2i, 2i+1$) with 0-indexed arrays.
9. MEMORY TRICK: "Children are double plus one/two."
10. IMPLEMENTATION IMPACT: Removes pointer overhead and gives $O(\log n)$ push/pop.

80. BST INORDER INVARIANT

1. FORMULA: For every node, all left keys $<$ node.key $<$ all right keys; inorder traversal is sorted.
2. CONCEPT: BST structure encodes sorted order.
3. INTUITION: Recursively, left subtree produces smaller values, then node, then larger values.
4. DSA CONNECTION: BST, TreeMap, binary search tree validation, kth element.
5. INTERVIEW PROBLEMS: Validate BST (LC 98), Kth Smallest in BST (LC 230), Recover BST (LC 99).
6. PATTERN RECOGNITION: Need sorted order from a tree or validate range constraints.
7. VISUALIZATION: Inorder of BST [left, root, right] outputs increasing sequence.
8. COMMON MISTAKES: Checking only immediate children instead of min/max ancestor bounds.
9. MEMORY TRICK: "Inorder turns BST into sorted array."
10. IMPLEMENTATION IMPACT: Enables $O(h)$ search and $O(n)$ sorted traversal without sorting.

81. SEGMENT TREE SIZE AND INTERVALS

1. FORMULA: Segment tree over n items uses at most $4n$ nodes; each query/update touches $O(\log n)$ levels.
2. CONCEPT: Recursively split array intervals in half.
3. INTUITION: The tree height is logarithmic and total nodes are bounded by a constant multiple of leaves.
4. DSA CONNECTION: Segment tree, range min/max/sum, lazy propagation.
5. INTERVIEW PROBLEMS: Range Sum Query Mutable (LC 307), Count of Smaller Numbers After Self (LC 315).
6. PATTERN RECOGNITION: Many online point updates and range queries.
7. VISUALIZATION: Root $[0, n-1]$, children $[0, \text{mid}]$ and $[\text{mid}+1, n-1]$.
8. COMMON MISTAKES: Building too small an array; lazy propagation mistakes on partial overlaps.
9. MEMORY TRICK: "Four n is the safe segment tree box."
10. IMPLEMENTATION IMPACT: Converts updates + queries from $O(n)$ each to $O(\log n)$.

82. FENWICK TREE PREFIX FORMULA

1. FORMULA: Update index i : $i += i \& -i$; query prefix: $i -= i \& -i$.
2. CONCEPT: Each index stores a range whose length is its lowbit.
3. INTUITION: Lowbit jumps move to the next responsible bucket or parent bucket.
4. DSA CONNECTION: Fenwick tree, prefix sums with updates, inversion counting.
5. INTERVIEW PROBLEMS: Count of Smaller Numbers After Self (LC 315), Range Sum Query Mutable (LC 307).
6. PATTERN RECOGNITION: Need point updates and prefix/range sum queries with compact code.
7. VISUALIZATION: Index 12 (1100) stores a block of length 4.
8. COMMON MISTAKES: Fenwick is usually 1-indexed; 0-indexed use needs care.
9. MEMORY TRICK: "Lowbit is bucket size."
10. IMPLEMENTATION IMPACT: $O(\log n)$ update/query with simpler implementation than segment tree for sums.

83. UNDIRECTED DEGREE SUM

1. FORMULA: $\text{sum degrees} = 2E$.
2. CONCEPT: Every undirected edge contributes 1 degree to each endpoint.
3. INTUITION: Counting edge endpoints counts each edge twice.
4. DSA CONNECTION: Graphs, Euler path, graph validation.
5. INTERVIEW PROBLEMS: Find Center of Star Graph (LC 1791), graph degree problems.
6. PATTERN RECOGNITION: Degree counts in an undirected graph.
7. VISUALIZATION: Edge (u, v) adds one mark at u and one at v .
8. COMMON MISTAKES: Forgetting self-loops contribute 2 in undirected degree.
9. MEMORY TRICK: "Edges have two ends."
10. IMPLEMENTATION IMPACT: Enables $O(n+E)$ degree-based algorithms and sanity checks.

84. DIRECTED DEGREE SUM

1. FORMULA: $\text{sum indegree} = \text{sum outdegree} = E$.
2. CONCEPT: Every directed edge leaves one node and enters one node.
3. INTUITION: Count edges by their tails or by their heads; both count each edge once.
4. DSA CONNECTION: Topological sort, Euler path, graph modeling.
5. INTERVIEW PROBLEMS: Course Schedule (LC 207), Reconstruct Itinerary (LC 332).

6. PATTERN RECOGNITION: Directed dependencies or prerequisites.
7. VISUALIZATION: $u \rightarrow v$ increments $out[u]$ and $in[v]$.
8. COMMON MISTAKES: Reversing edge direction in prerequisite problems.
9. MEMORY TRICK: "Every arrow has one tail and one head."
10. IMPLEMENTATION IMPACT: Supports $O(V+E)$ topological processing.

85. COMPLETE GRAPH EDGE COUNT

1. FORMULA: Undirected complete graph: $E = n(n-1)/2$; directed without self-loops: $E = n(n-1)$.
2. CONCEPT: Every pair of vertices is connected.
3. INTUITION: Choose unordered pairs for undirected; ordered pairs for directed.
4. DSA CONNECTION: Graph density, adjacency matrix, MST, pair counting.
5. INTERVIEW PROBLEMS: Network connection variants, pairwise distance graphs.
6. PATTERN RECOGNITION: All pairs of nodes/items interact.
7. VISUALIZATION: 4 nodes have 6 undirected pair edges.
8. COMMON MISTAKES: Double-counting undirected edges.
9. MEMORY TRICK: "Complete graph is choose two."
10. IMPLEMENTATION IMPACT: Identifies $O(n^2)$ unavoidable pair generation in dense graphs.

86. CONNECTED COMPONENTS

1. FORMULA: In an undirected graph, each successful union reduces component count by 1; final components = $n - \text{successfulUnions}$.
2. CONCEPT: Components are maximal connected groups.
3. INTUITION: Joining two previously separate groups merges them into one.
4. DSA CONNECTION: Union Find, DFS/BFS, graph connectivity.
5. INTERVIEW PROBLEMS: Number of Connected Components (LC 323), Number of Provinces (LC 547), Accounts Merge (LC 721).
6. PATTERN RECOGNITION: Groups, connectivity, equivalence relations.
7. VISUALIZATION: Start with isolated dots; every bridge between groups reduces island count.
8. COMMON MISTAKES: Decrementing component count when unioning nodes already in same component.
9. MEMORY TRICK: "Only new bridges reduce islands."
10. IMPLEMENTATION IMPACT: DSU gives near $O(1)$ amortized connectivity operations.

87. DAG TOPOLOGICAL INVARIANT

1. FORMULA: A graph is a DAG iff it has a topological order; every edge $u \rightarrow v$ has $pos[u] < pos[v]$.
2. CONCEPT: Dependencies can be ordered only when there is no cycle.
3. INTUITION: A cycle would require some node to come before itself.
4. DSA CONNECTION: Graphs, scheduling, DP on DAGs, build systems.
5. INTERVIEW PROBLEMS: Course Schedule (LC 207), Course Schedule II (LC 210), Alien Dictionary.
6. PATTERN RECOGNITION: Prerequisites, dependencies, "must happen before".
7. VISUALIZATION: Remove zero-indegree nodes layer by layer.
8. COMMON MISTAKES: Treating visited DFS alone as cycle detection without recursion-stack state.
9. MEMORY TRICK: "DAG means dependency line has no loop."
10. IMPLEMENTATION IMPACT: Topological sort turns recursive dependency resolution into $O(V+E)$.

88. BIPARTITE GRAPH INVARIANT

1. FORMULA: A graph is bipartite iff it has no odd cycle.
2. CONCEPT: Vertices can be colored with two colors so every edge crosses colors.
3. INTUITION: Alternating colors around a cycle works only for even length; odd length creates a conflict.
4. DSA CONNECTION: BFS/DFS coloring, matching, graph validation.
5. INTERVIEW PROBLEMS: Is Graph Bipartite? (LC 785), Possible Bipartition (LC 886).
6. PATTERN RECOGNITION: Split into two groups with dislikes/conflicts.
7. VISUALIZATION: A triangle cannot be 2-colored because the third edge connects same colors.
8. COMMON MISTAKES: Not checking disconnected components.
9. MEMORY TRICK: "Odd cycle breaks two teams."
10. IMPLEMENTATION IMPACT: $O(V+E)$ coloring replaces exponential group assignment.

89. EULER PATH AND CIRCUIT

1. FORMULA: Undirected Euler circuit: all degrees even; Euler path: exactly 0 or 2 odd-degree vertices, and graph connected over nonzero-degree nodes.
2. CONCEPT: Traverse every edge exactly once.
3. INTUITION: Every time you enter a middle vertex, you must leave using a different unused edge; odd vertices can only be endpoints.
4. DSA CONNECTION: Graph traversal, Hierholzer algorithm, itinerary reconstruction.
5. INTERVIEW PROBLEMS: Reconstruct Itinerary (LC 332), Valid Arrangement of Pairs (LC 2097).
6. PATTERN RECOGNITION: Use every edge/ticket/pair exactly once.
7. VISUALIZATION: Edges pair up as enter/exit at internal vertices.
8. COMMON MISTAKES: Confusing Euler path (edges once) with Hamiltonian path (vertices once).

9. MEMORY TRICK: "Euler cares about edges."

10. IMPLEMENTATION IMPACT: Degree checks and Hierholzer solve in $O(E \log E)$ or $O(E)$.

90. MST CUT AND CYCLE PROPERTIES

1. FORMULA: Cut property: the lightest edge crossing any cut is safe; cycle property: the heaviest edge on a cycle is not needed in some MST.

2. CONCEPT: MST choices are justified by cuts and cycles.

3. INTUITION: If a cheaper crossing edge exists, replacing a heavier one keeps connectivity and reduces cost.

4. DSA CONNECTION: Graphs, Kruskal, Prim, Union Find.

5. INTERVIEW PROBLEMS: Min Cost to Connect Points (LC 1584), Connecting Cities With Minimum Cost.

6. PATTERN RECOGNITION: Need connect all nodes with minimum total edge weight.

7. VISUALIZATION: Among edges crossing a partition, pick the cheapest bridge.

8. COMMON MISTAKES: Applying MST to shortest path; MST minimizes total network, not path distance.

9. MEMORY TRICK: "Cheapest bridge across a cut is safe."

10. IMPLEMENTATION IMPACT: Greedy MST algorithms run in $O(E \log E)$ instead of checking all spanning trees.

91. SHORTEST PATH RELAXATION

1. FORMULA: If $\text{dist}[v] > \text{dist}[u] + w(u,v)$, set $\text{dist}[v] = \text{dist}[u] + w(u,v)$.

2. CONCEPT: Improve the best known distance through an edge.

3. INTUITION: A path to u plus edge $u \rightarrow v$ is a candidate path to v .

4. DSA CONNECTION: Dijkstra, Bellman-Ford, DAG shortest path.

5. INTERVIEW PROBLEMS: Network Delay Time (LC 743), Cheapest Flights Within K Stops (LC 787), Path With Minimum Effort (LC 1631).

6. PATTERN RECOGNITION: Weighted graph, minimum cost/distance.

7. VISUALIZATION: Distance labels shrink when a better route is discovered.

8. COMMON MISTAKES: Using Dijkstra with negative edges; not skipping stale priority queue entries.

9. MEMORY TRICK: "Relax means try a better route."

10. IMPLEMENTATION IMPACT: Gives $O(E \log V)$ Dijkstra with heap for nonnegative weights.

PART 8: BINARY SEARCH MATHEMATICS

92. SEARCH SPACE HALVING

1. FORMULA: After t halvings, remaining size is $n / 2^t$; need $t \geq \text{ceil}(\log_2 n)$.

2. CONCEPT: Binary search discards half the candidates each step.

3. INTUITION: Repeated division by 2 reaches 1 after logarithmically many cuts.

4. DSA CONNECTION: Binary search, balanced trees, heaps, divide and conquer.

5. INTERVIEW PROBLEMS: Binary Search (LC 704), Search in Rotated Sorted Array (LC 33).

6. PATTERN RECOGNITION: Sorted data or monotonic decision.

7. VISUALIZATION: 1024 elements $\rightarrow$ 512 $\rightarrow$ 256 $\rightarrow$... $\rightarrow$ 1 in 10 cuts.

8. COMMON MISTAKES: Infinite loops from wrong boundary updates.

9. MEMORY TRICK: "Half, half, half means log."

10. IMPLEMENTATION IMPACT: Converts $O(n)$ search to $O(\log n)$.

93. MONOTONIC PREDICATE

1. FORMULA: Find first i where $P(i)=\text{true}$ when sequence is false false ... true true.

2. CONCEPT: Binary search can locate the boundary between false and true.

3. INTUITION: Testing mid tells which side still contains the transition.

4. DSA CONNECTION: Binary search on answer, lower bound, feasibility checks.

5. INTERVIEW PROBLEMS: First Bad Version (LC 278), Find Minimum in Rotated Sorted Array (LC 153).

6. PATTERN RECOGNITION: "Minimum x such that possible" or "first position satisfying condition".

7. VISUALIZATION: F F F F T T T; binary search finds the first T.

8. COMMON MISTAKES: Predicate is not truly monotonic; binary search then gives nonsense.

9. MEMORY TRICK: "Binary search finds a cliff."

10. IMPLEMENTATION IMPACT: Turns answer search over huge numeric range into $O(\log \text{range} * \text{checkCost})$.

94. LOWER BOUND AND UPPER BOUND

1. FORMULA: $\text{lower_bound}(x)$ first index with $\text{arr}[i] \geq x$; $\text{upper_bound}(x)$ first index with $\text{arr}[i] > x$.

2. CONCEPT: Boundary positions in sorted arrays.

3. INTUITION: Sorted order makes the predicate monotonic.

4. DSA CONNECTION: Arrays, binary search, frequency counting, LIS.

5. INTERVIEW PROBLEMS: Find First and Last Position (LC 34), Search Insert Position (LC 35), Longest Increasing Subsequence (LC 300).

6. PATTERN RECOGNITION: Need insertion point, first/last occurrence, count of values in a range.

7. VISUALIZATION: In $[1, 2, 2, 2, 5]$, lower bound of 2 is 1, upper bound is 4.

8. COMMON MISTAKES: Using $\leq$ in the wrong branch; not clearly choosing first $\geq$ vs first $>$.

9. MEMORY TRICK: "Lower allows equal; upper goes past equal."

10. IMPLEMENTATION IMPACT: Counts duplicates in $O(\log n)$ instead of scanning.

95. BINARY SEARCH ON ANSWER

1. FORMULA: Search minimal feasible x where $\text{can}(x)$ is monotonic.
2. CONCEPT: Search the value domain, not an array index.
3. INTUITION: If capacity/time/limit x works, any larger relaxed value often also works.
4. DSA CONNECTION: Greedy feasibility, arrays, scheduling, graph thresholds.
5. INTERVIEW PROBLEMS: Koko Eating Bananas (LC 875), Capacity to Ship Packages (LC 1011), Split Array Largest Sum (LC 410).
6. PATTERN RECOGNITION: "Minimize maximum", "smallest capacity", "minimum speed", "can finish within".
7. VISUALIZATION: Speeds: 1 2 3 4 5 6; maybe F F F T T T.
8. COMMON MISTAKES: Weak lower/upper bounds; feasibility check accidentally non-monotonic.
9. MEMORY TRICK: "Ask yes/no about the answer."
10. IMPLEMENTATION IMPACT: Replaces trying every possible answer with logarithmic search.

96. REAL-VALUED BINARY SEARCH

1. FORMULA: Repeat $\text{mid} = (\text{lo} + \text{hi})/2$ for fixed iterations or until $\text{hi} - \text{lo} < \text{eps}$.
2. CONCEPT: Approximate a real-valued boundary.
3. INTUITION: Continuous monotonic functions still have a left/right side around the target.
4. DSA CONNECTION: Geometry, numerical optimization, probability thresholds.
5. INTERVIEW PROBLEMS: Minimize Max Distance to Gas Station (LC 774), square root variants.
6. PATTERN RECOGNITION: Answer is decimal and predicate is monotonic.
7. VISUALIZATION: Zoom on a number line interval until it is tiny.
8. COMMON MISTAKES: Infinite loop from exact equality on doubles; use iteration count/epsilon.
9. MEMORY TRICK: "Doubles need tolerance, not equality."
10. IMPLEMENTATION IMPACT: Solves continuous search in $O(\log((\text{hi}-\text{lo})/\text{eps}) * \text{checkCost})$.

97. EXPONENTIAL SEARCH BOUNDS

1. FORMULA: Try 1,2,4,8,... until predicate becomes true, then binary search previous range.
2. CONCEPT: Discover an unknown upper bound logarithmically.
3. INTUITION: Doubling reaches any finite answer A in $\text{ceil}(\log_2 A)$ steps.
4. DSA CONNECTION: Infinite arrays, unbounded answer search, streams.
5. INTERVIEW PROBLEMS: Search in a Sorted Array of Unknown Size (LC 702), unbounded capacity problems.
6. PATTERN RECOGNITION: No explicit upper bound but predicate eventually becomes true.
7. VISUALIZATION: 1 -> 2 -> 4 -> 8 -> 16; if answer is 11, search [8,16].
8. COMMON MISTAKES: Overflow during doubling; cap at max value.
9. MEMORY TRICK: "Double to find the wall."
10. IMPLEMENTATION IMPACT: Avoids arbitrary huge bounds while keeping $O(\log \text{answer})$.

98. TERNARY SEARCH FOR UNIMODAL FUNCTIONS

1. FORMULA: For unimodal f , compare $f(m_1)$ and $f(m_2)$ where $m_1 = \text{lo} + (\text{hi}-\text{lo})/3$, $m_2 = \text{hi} - (\text{hi}-\text{lo})/3$.
2. CONCEPT: A single peak or valley allows discarding one third of the range.
3. INTUITION: If $f(m_1) < f(m_2)$ in a maximum search, the peak is to the right of m_1 .
4. DSA CONNECTION: Optimization, geometry, math search.
5. INTERVIEW PROBLEMS: Rare in LeetCode interviews, common in Codeforces numeric optimization.
6. PATTERN RECOGNITION: Function decreases then increases, or increases then decreases, with no multiple local extrema.
7. VISUALIZATION: A hill shape has one summit; compare two interior points to locate the summit side.
8. COMMON MISTAKES: Using ternary search on non-unimodal functions.
9. MEMORY TRICK: "One hill, cut thirds."
10. IMPLEMENTATION IMPACT: Optimizes continuous/discrete unimodal objectives without derivative calculus.

PART 9: DYNAMIC PROGRAMMING MATHEMATICS

99. RECURRENCE RELATION

1. FORMULA: $\text{dp}[\text{state}] = \text{combine}(\text{dp}[\text{previous states}])$.
2. CONCEPT: Define an answer in terms of smaller answers.
3. INTUITION: If optimal solution ends with a choice, remove that choice and solve the remaining subproblem.
4. DSA CONNECTION: DP, recursion, memoization, graph DAG DP.
5. INTERVIEW PROBLEMS: Climbing Stairs (LC 70), House Robber (LC 198), Coin Change (LC 322).
6. PATTERN RECOGNITION: Overlapping subproblems and choices that build from smaller cases.
7. VISUALIZATION: A DP table fills cells from already-known neighbors.
8. COMMON MISTAKES: Vague state definition; transition uses future/uncomputed states.
9. MEMORY TRICK: "State + transition + base case."
10. IMPLEMENTATION IMPACT: Reduces exponential recursion to polynomial states.

100. DP COMPLEXITY FORMULA

1. FORMULA: $\text{time} = \text{number_of_states} * \text{transition_cost}$; $\text{space} = \text{number_of_states} * \text{value_size}$.
2. CONCEPT: DP cost is table size times work per cell.
3. INTUITION: Each state is computed once in memoized/tabulated DP.
4. DSA CONNECTION: All DP, memoization, bitmask DP, tree DP.
5. INTERVIEW PROBLEMS: All DP problems, especially Coin Change (LC 322), Edit Distance (LC 72).
6. PATTERN RECOGNITION: Before coding DP, count dimensions and choices.
7. VISUALIZATION: $\text{dp}[i][j]$ table has $n*m$ cells; if each scans k , time is $O(nmk)$.
8. COMMON MISTAKES: Saying $O(n)$ because there is one loop in recursion; hidden states multiply.
9. MEMORY TRICK: "Cells times choices."
10. IMPLEMENTATION IMPACT: Guides optimization from 2D to 1D or from $O(n^2)$ transitions to optimized structures.

101. FIBONACCI RECURRENCE

1. FORMULA: $F(0)=0$, $F(1)=1$, $F(n)=F(n-1)+F(n-2)$.
2. CONCEPT: Current value depends on the two previous values.
3. INTUITION: Many count problems end with one of two last-step choices.
4. DSA CONNECTION: DP, recursion, matrix exponentiation.
5. INTERVIEW PROBLEMS: Climbing Stairs (LC 70), Fibonacci Number (LC 509), Min Cost Climbing Stairs (LC 746).
6. PATTERN RECOGNITION: Ways to reach step n using steps of size 1 or 2.
7. VISUALIZATION: To reach stair n , come from $n-1$ by one step or $n-2$ by two steps.
8. COMMON MISTAKES: Exponential recursion without memoization.
9. MEMORY TRICK: "Last jump was 1 or 2."
10. IMPLEMENTATION IMPACT: Memoization changes $O(2^n)$ recursion to $O(n)$, $O(1)$ space with rolling variables.

102. KNAPSACK RECURRENCE

1. FORMULA: $\text{dp}[i][w] = \max(\text{dp}[i-1][w], \text{value}[i] + \text{dp}[i-1][w-\text{weight}[i]])$ when $\text{weight}[i] \leq w$.
2. CONCEPT: For each item, choose take or skip.
3. INTUITION: An optimal solution either excludes the item or includes it and solves remaining capacity.
4. DSA CONNECTION: DP, subset sum, resource allocation.
5. INTERVIEW PROBLEMS: Partition Equal Subset Sum (LC 416), Target Sum (LC 494), Last Stone Weight II (LC 1049).
6. PATTERN RECOGNITION: Choose subset under capacity/target constraint.
7. VISUALIZATION: Capacity axis columns; each item row updates reachable/best capacities.
8. COMMON MISTAKES: For 0/1 knapsack, iterating capacity upward in 1D and reusing the same item multiple times.
9. MEMORY TRICK: "Skip or take."
10. IMPLEMENTATION IMPACT: Converts exponential subset enumeration to $O(nW)$.

103. LONGEST COMMON SUBSEQUENCE

1. FORMULA: If $a[i-1]=b[j-1]$, $\text{dp}[i][j]=1+\text{dp}[i-1][j-1]$; else $\max(\text{dp}[i-1][j], \text{dp}[i][j-1])$.
2. CONCEPT: Longest sequence preserving order in both strings.
3. INTUITION: Matching last characters can be paired; otherwise drop one last character.
4. DSA CONNECTION: DP, strings, diff algorithms.
5. INTERVIEW PROBLEMS: Longest Common Subsequence (LC 1143), Delete Operation for Two Strings (LC 583).
6. PATTERN RECOGNITION: Two strings/arrays and order matters but contiguity does not.
7. VISUALIZATION: DP grid compares prefixes of both strings.
8. COMMON MISTAKES: Confusing subsequence with substring; substring requires contiguity and reset on mismatch.
9. MEMORY TRICK: "Match diagonally, otherwise drop one side."
10. IMPLEMENTATION IMPACT: $O(nm)$ DP replaces exponential subsequence enumeration.

104. EDIT DISTANCE

1. FORMULA: $\text{dp}[i][j] = \min(\text{delete}, \text{insert}, \text{replace})$ with cost 0 if current chars match else 1.
2. CONCEPT: Minimum operations to transform one prefix into another.
3. INTUITION: Last operation must be insert, delete, replace, or match.
4. DSA CONNECTION: DP, strings, NLP/search similarity.
5. INTERVIEW PROBLEMS: Edit Distance (LC 72), One Edit Distance (LC 161).
6. PATTERN RECOGNITION: Minimum edits/conversions between two strings.
7. VISUALIZATION: Grid cell (i,j) means transform first i chars into first j chars.
8. COMMON MISTAKES: Wrong base cases: converting length i to empty costs i .
9. MEMORY TRICK: "Last edit explains the cell."
10. IMPLEMENTATION IMPACT: $O(nm)$ DP instead of branching over all edit sequences.

105. GRID PATH COUNTING

1. FORMULA: Without obstacles, paths from top-left to bottom-right are $C((m-1)+(n-1), m-1)$.
2. CONCEPT: A path is an ordering of down and right moves.
3. INTUITION: Choose which positions among all moves are down moves.
4. DSA CONNECTION: DP, combinatorics, grid traversal.
5. INTERVIEW PROBLEMS: Unique Paths (LC 62), Unique Paths II (LC 63).
6. PATTERN RECOGNITION: Move only right/down on a grid.
7. VISUALIZATION: In a 3x3 grid, need 2 rights and 2 downs: $C(4,2)=6$.
8. COMMON MISTAKES: Formula fails with obstacles; then use DP.
9. MEMORY TRICK: "Grid path is choose positions for downs."
10. IMPLEMENTATION IMPACT: $O(1)/O(\min(m,n))$ combinatorics for obstacle-free grids; DP for obstacles.

106. LONGEST INCREASING SUBSEQUENCE

1. FORMULA: DP: $dp[i]=1+\max(dp[j])$ for $j<i$ and $a[j]<a[i]$; optimized tails: $tails[len] = \text{smallest tail of length } len+1$.
2. CONCEPT: Find longest order-preserving increasing sequence.
3. INTUITION: Smaller tail is always better because it leaves more room for future values.
4. DSA CONNECTION: DP, binary search, patience sorting.
5. INTERVIEW PROBLEMS: Longest Increasing Subsequence (LC 300), Russian Doll Envelopes (LC 354).
6. PATTERN RECOGNITION: Need increasing subsequence, not necessarily contiguous.
7. VISUALIZATION: Place each number on the leftmost pile whose top is $\geq$ number.
8. COMMON MISTAKES: Using non-strict comparison when problem requires strictly increasing.
9. MEMORY TRICK: "Small tails keep options open."
10. IMPLEMENTATION IMPACT: Improves $O(n^2)$ DP to $O(n \log n)$.

107. INTERVAL DP

1. FORMULA: $dp[l][r] = \text{best over } k \text{ of } \text{combine}(dp[l][k], dp[k+1][r], \text{cost}(l,k,r))$.
2. CONCEPT: Solve every interval by splitting it.
3. INTUITION: The last operation often splits the interval into independent left/right parts.
4. DSA CONNECTION: DP, parsing, matrix chain multiplication, burst balloons.
5. INTERVIEW PROBLEMS: Burst Balloons (LC 312), Minimum Cost to Cut a Stick (LC 1547), Palindrome Partitioning II (LC 132).
6. PATTERN RECOGNITION: Operations remove/merge/cut inside a contiguous interval.
7. VISUALIZATION: Choose the last balloon k ; left and right intervals are solved independently.
8. COMMON MISTAKES: Filling intervals in wrong order; shorter intervals must be computed first.
9. MEMORY TRICK: "Pick the last split."
10. IMPLEMENTATION IMPACT: Turns factorial operation orders into $O(n^3)$ DP, sometimes optimizable.

108. TREE DP AND REROOTING

1. FORMULA: Down DP combines children; reroot transition moves contribution across edge: answer at child from answer at parent minus child contribution plus parent-side contribution.
2. CONCEPT: Compute values for every possible root in a tree.
3. INTUITION: First DFS knows subtree answers; second DFS transfers the root viewpoint across edges.
4. DSA CONNECTION: Trees, DFS, DP, rerooting.
5. INTERVIEW PROBLEMS: Sum of Distances in Tree (LC 834), Binary Tree Maximum Path Sum (LC 124).
6. PATTERN RECOGNITION: Need answer for every node as root or global path using subtree information.
7. VISUALIZATION: Moving root from u to child v : nodes in v subtree get closer, all others get farther.
8. COMMON MISTAKES: Mixing downward-only value with through-parent value.
9. MEMORY TRICK: "First collect, then redistribute."
10. IMPLEMENTATION IMPACT: Computes all-root answers in $O(n)$ instead of $O(n^2)$ DFS from each node.

109. DIGIT DP

1. FORMULA: $dp[pos][tight][state]$ counts valid prefixes under an upper-bound prefix constraint.
2. CONCEPT: Count numbers digit by digit without enumerating them.
3. INTUITION: tight records whether the current prefix equals the bound; if not tight, remaining digits are free.
4. DSA CONNECTION: DP, number theory, counting under constraints.
5. INTERVIEW PROBLEMS: Number of Digit One (LC 233), Count Numbers with Unique Digits (LC 357), Digit DP competitive tasks.
6. PATTERN RECOGNITION: Count integers in $[0,N]$ satisfying digit properties.
7. VISUALIZATION: While matching $N=527$, at prefix 52 next digit is limited; at prefix 51, future digits are free.
8. COMMON MISTAKES: Mishandling leading zeros and inclusive bounds.
9. MEMORY TRICK: "Tight means still glued to the bound."
10. IMPLEMENTATION IMPACT: Counts up to 10^{18} in $O(\text{number_of_digits} * \text{states} * 10)$.

110. PROBABILITY DP

1. FORMULA: $dp[state] = \sum P(choice) * dp[next_state]$ for expected/future value, or sum probabilities for distribution.
2. CONCEPT: Probability transitions average over possible random outcomes.
3. INTUITION: Law of total probability splits outcomes by next random event.
4. DSA CONNECTION: DP, Markov chains, games, randomized processes.
5. INTERVIEW PROBLEMS: New 21 Game (LC 837), Knight Probability in Chessboard (LC 688), Soup Servings (LC 808).
6. PATTERN RECOGNITION: Random process with repeated states and probability of success.
7. VISUALIZATION: Knight probability at a square is average of probabilities from valid previous/next moves.
8. COMMON MISTAKES: Not normalizing by number/probability of moves; double-counting invalid transitions.
9. MEMORY TRICK: "Probability DP is weighted sum of futures."
10. IMPLEMENTATION IMPACT: Replaces Monte Carlo simulation with exact polynomial DP.

PART 10: COMPLEXITY ANALYSIS

111. BIG O DEFINITION

1. FORMULA: $f(n) = O(g(n))$ if there exist constants c, n_0 such that $f(n) \leq c * g(n)$ for all $n \geq n_0$.
2. CONCEPT: Upper-bound growth rate for large inputs.
3. INTUITION: Ignore constant factors and smaller terms because dominant growth controls scalability.
4. DSA CONNECTION: All algorithm analysis.
5. INTERVIEW PROBLEMS: Every interview problem discussion.
6. PATTERN RECOGNITION: Need to explain runtime/space as input grows.
7. VISUALIZATION: $3n^2 + 10n + 5$ behaves like n^2 for large n .
8. COMMON MISTAKES: Calling $O(n)$ "exact time"; Big O is an upper bound.
9. MEMORY TRICK: "Keep the fastest-growing term."
10. IMPLEMENTATION IMPACT: Helps choose algorithms that fit constraints.

112. COMMON GROWTH ORDER

1. FORMULA: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$.
2. CONCEPT: Standard scalability hierarchy.
3. INTUITION: Polynomial, exponential, and factorial growth separate dramatically as n increases.
4. DSA CONNECTION: Complexity analysis, algorithm selection.
5. INTERVIEW PROBLEMS: All performance discussions.
6. PATTERN RECOGNITION: Compare constraints to feasible growth.
7. VISUALIZATION: For $n=1000$, $n^2=1,000,000$, but 2^n is impossible.
8. COMMON MISTAKES: Thinking $O(n \log n)$ and $O(n^2)$ are close for large n .
9. MEMORY TRICK: "Constant, log, linear, sort, square, subset, permutation."
10. IMPLEMENTATION IMPACT: Guides whether sorting, DP, or pruning is acceptable.

113. LOGARITHMIC GROWTH

1. FORMULA: Repeated division by b : iterations $\text{ceil}(\log_b n)$.
2. CONCEPT: Logarithms count how many times input shrinks by a constant factor.
3. INTUITION: After t steps, size is n/b^t ; solve $n/b^t \leq 1$.
4. DSA CONNECTION: Binary search, balanced trees, heaps, divide and conquer.
5. INTERVIEW PROBLEMS: Binary Search (LC 704), heap operations, TreeMap operations.
6. PATTERN RECOGNITION: Loop does $n/2$ or search interval halves.
7. VISUALIZATION: while $(n > 1)$ $n /= 2$ runs about $\log_2 n$ times.
8. COMMON MISTAKES: Calling for $i*=2$ loops $O(n)$; they are $O(\log n)$.
9. MEMORY TRICK: "Multiply or divide loop counter means log."
10. IMPLEMENTATION IMPACT: Supports using balanced structures for large data.

114. ARITHMETIC SERIES IN NESTED LOOPS

1. FORMULA: $\sum_{i=1..n} i = O(n^2)$.
2. CONCEPT: Triangular nested loops are quadratic.
3. INTUITION: Total work is half of an n by n square, still quadratic.
4. DSA CONNECTION: Pair enumeration, brute-force subarrays, sorting analysis.
5. INTERVIEW PROBLEMS: 3Sum brute force, subarray enumeration problems.
6. PATTERN RECOGNITION: Inner loop length depends linearly on outer index.
7. VISUALIZATION: Rows of lengths $1, 2, 3, \dots, n$ form a triangle.
8. COMMON MISTAKES: Thinking it is $O(n)$ because there is one nested loop but not full square.
9. MEMORY TRICK: "Triangle is still square order."
10. IMPLEMENTATION IMPACT: Motivates prefix sums, hash maps, sorting, or two pointers to beat $O(n^2)$.

115. GEOMETRIC SERIES IN ALGORITHMS

1. FORMULA: $n + n/2 + n/4 + \dots < 2n$.

2. CONCEPT: Work that halves each level sums to linear.
3. INTUITION: The tail never exceeds the first term times a constant.
4. DSA CONNECTION: Divide and conquer, recursion trees, amortized resizing.
5. INTERVIEW PROBLEMS: Binary search variants, tree traversals, dynamic array resizing analysis.
6. PATTERN RECOGNITION: Each level has shrinking total work by constant factor.
7. VISUALIZATION: $16 + 8 + 4 + 2 + 1 = 31 < 32$.
8. COMMON MISTAKES: Assuming all recursive algorithms are $O(n \log n)$; some geometric sums are $O(n)$.
9. MEMORY TRICK: "Halving sum is less than double."
10. IMPLEMENTATION IMPACT: Explains linear amortized behavior and divide-and-conquer costs.

116. RECURSION TREE

1. FORMULA: Total time = sum of work over all recursion levels.
2. CONCEPT: Expand a recurrence as a tree of subproblems.
3. INTUITION: Each recursive call contributes local work; add across levels until base cases.
4. DSA CONNECTION: Merge sort, quicksort, DFS recursion, divide and conquer.
5. INTERVIEW PROBLEMS: Sort List (LC 148), Merge k Sorted Lists (LC 23), recursive tree problems.
6. PATTERN RECOGNITION: Recurrence has multiple calls or shrinking sizes.
7. VISUALIZATION: Merge sort: each level totals $O(n)$, and there are $O(\log n)$ levels.
8. COMMON MISTAKES: Counting only one branch instead of all branches.
9. MEMORY TRICK: "Cost per level times number of levels."
10. IMPLEMENTATION IMPACT: Gives accurate complexity before optimizing recursion.

117. MASTER THEOREM

1. FORMULA: For $T(n) = aT(n/b) + f(n)$, compare $f(n)$ with $n^{\log_b a}$.
2. CONCEPT: Solves common divide-and-conquer recurrences.
3. INTUITION: $n^{\log_b a}$ represents total leaf pressure; $f(n)$ represents per-level combine pressure.
4. DSA CONNECTION: Sorting, divide and conquer, binary splitting algorithms.
5. INTERVIEW PROBLEMS: Merge Sort, Binary Search, Karatsuba-style discussions.
6. PATTERN RECOGNITION: Same-size subproblems plus combine work.
7. VISUALIZATION: Merge sort: $a=2, b=2, f(n)=n$; compare with $n^{\log_2 2} = n$, so $O(n \log n)$.
8. COMMON MISTAKES: Applying Master theorem to uneven splits or non-polynomial irregular recurrences.
9. MEMORY TRICK: "Compare combine work to leaf work."
10. IMPLEMENTATION IMPACT: Quickly classifies divide-and-conquer performance.

118. AMORTIZED ANALYSIS

1. FORMULA: Amortized cost = total cost over sequence / number of operations.
2. CONCEPT: Occasional expensive operations can average out over many cheap ones.
3. INTUITION: Expensive resize/rebuild is paid for by many previous simple operations.
4. DSA CONNECTION: Dynamic arrays, hash tables, stacks, queues, DSU.
5. INTERVIEW PROBLEMS: Min Stack (LC 155), Implement Queue using Stacks (LC 232), RandomizedSet (LC 380).
6. PATTERN RECOGNITION: A data structure occasionally resizes, rebuilds, or compresses paths.
7. VISUALIZATION: Dynamic array doubles capacity; copying after sizes 1,2,4,8 sums below $2n$.
8. COMMON MISTAKES: Confusing amortized $O(1)$ with worst-case $O(1)$.
9. MEMORY TRICK: "Spread the spike over the sequence."
10. IMPLEMENTATION IMPACT: Justifies practical $O(1)$ append/hash operations.

119. SORTING LOWER BOUND

1. FORMULA: Comparison sorting needs $\Omega(n \log n)$ comparisons in the worst case.
2. CONCEPT: Any comparison sort must distinguish among $n!$ possible orders.
3. INTUITION: A comparison decision tree with height h has at most 2^h leaves; need $2^h \geq n!$.
4. DSA CONNECTION: Sorting, selection, lower-bound reasoning.
5. INTERVIEW PROBLEMS: Sort an Array (LC 912), Kth Largest Element (LC 215).
6. PATTERN RECOGNITION: If only comparisons are allowed, do not expect better than $O(n \log n)$ full sort.
7. VISUALIZATION: Every possible permutation must end at a different leaf in the comparison tree.
8. COMMON MISTAKES: Sorting when selection/top-k would solve in $O(n)$ or $O(n \log k)$.
9. MEMORY TRICK: "Sorting must identify one of $n!$ orders."
10. IMPLEMENTATION IMPACT: Motivates heap/quickselect/counting sort when full comparison sorting is unnecessary or keys are bounded.

120. CONSTRAINT-TO-COMPLEXITY HEURISTIC

1. FORMULA: Rough feasible operations: 10^8 simple operations per second in compiled languages; interview Java usually aim lower.
2. CONCEPT: Input size suggests acceptable complexity.
3. INTUITION: Algorithms must fit both asymptotic growth and constant factors.
4. DSA CONNECTION: Choosing between brute force, DP, sorting, greedy, graph algorithms.
5. INTERVIEW PROBLEMS: All timed coding assessments.

6. PATTERN RECOGNITION: Read constraints before choosing approach.
7. VISUALIZATION: $n \leq 20$ may allow 2^n ; $n \leq 10^5$ usually needs $O(n \log n)$ or $O(n)$.
8. COMMON MISTAKES: Designing $O(n^2)$ for $n=10^5$.
9. MEMORY TRICK: "Constraints whisper the algorithm."
10. IMPLEMENTATION IMPACT: Prevents overengineering small constraints and under-optimizing large ones.

121. HASH TABLE EXPECTED COMPLEXITY

1. FORMULA: Expected $O(1)$ insert/find/delete under good hashing and bounded load factor; worst-case $O(n)$.
2. CONCEPT: Hashing spreads keys into buckets.
3. INTUITION: With uniform distribution, each bucket has constant expected length.
4. DSA CONNECTION: HashMap, HashSet, frequency counting, prefix maps.
5. INTERVIEW PROBLEMS: Two Sum (LC 1), Longest Consecutive Sequence (LC 128), Subarray Sum Equals K (LC 560).
6. PATTERN RECOGNITION: Need fast membership, frequency, or first-seen lookup.
7. VISUALIZATION: Keys are balls thrown into many buckets; low load means few collisions.
8. COMMON MISTAKES: Ignoring worst-case collision attacks in system-level settings; mutating keys after insertion.
9. MEMORY TRICK: "HashMap trades order for average $O(1)$."
10. IMPLEMENTATION IMPACT: Replaces nested searches with single-pass lookup patterns.

122. NUMERIC OVERFLOW BOUNDS

1. FORMULA: 32-bit signed int max $2^{31} - 1$; 64-bit signed long max $2^{63} - 1$.
2. CONCEPT: Fixed-width integers wrap or overflow beyond their range.
3. INTUITION: Bits store finite states; arithmetic can exceed representable values.
4. DSA CONNECTION: Binary search midpoints, products, sums, modular arithmetic.
5. INTERVIEW PROBLEMS: Reverse Integer (LC 7), Pow(x,n) (LC 50), mySqrt (LC 69).
6. PATTERN RECOGNITION: Multiplication of values up to 10^9 , summing many ints, midpoint calculation.
7. VISUALIZATION: $\text{mid} = (\text{lo} + \text{hi}) / 2$ can overflow; $\text{lo} + (\text{hi} - \text{lo})/2$ is safe.
8. COMMON MISTAKES: Casting after multiplication; cast before: $(\text{long})a * b$.
9. MEMORY TRICK: "Widen before multiply."
10. IMPLEMENTATION IMPACT: Prevents silent wrong answers in otherwise correct algorithms.

FINAL RECOGNITION MAP

Problem smell	Mathematical core	Typical optimal tool
Many range sums	Prefix difference	Prefix sum / Fenwick / segment tree
Many range updates	Telescoping boundaries	Difference array / lazy segment tree
Divisibility of subarray sum	Congruent prefixes	Prefix modulo frequency map
Count arrangements	Product rule, factorials, nCr	Combinatorics / DP
Duplicate existence by range	Pigeonhole	Floyd cycle / binary search on value
Unique among pairs	XOR cancellation	XOR scan
Small visited set	Subsets = bit masks	Bitmask DP
Points and turns	Cross product sign	Convex hull / intersection
Tree validity	Edges = nodes - 1 + connected	DFS / DSU
Dependencies	DAG topological invariant	Topological sort
Minimum feasible capacity	Monotonic predicate	Binary search on answer
Recursive overlapping choices	Recurrence relation	DP
Divide and conquer recurrence	Recursion tree	Master theorem

CLOSING PRINCIPLE

Most hard DSA problems become manageable when you translate the story into one of these mathematical objects: a prefix, a remainder class, a set, a bit mask, a graph invariant, a monotonic predicate, or a recurrence. The interview skill is not memorizing isolated formulas; it is recognizing which invariant is hiding inside the problem statement.